A

Industrial-Oriented Mini Project

On

HIERARCHICAL STROKE-DENSITY PROFILING FOR OFFLINE AUTOGRAPH IDENTITY CONFIRMATION

(Submitted in partial fulfilment of the requirements for the award of Degree)

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING

By

Malka Divya Sri (237R1A05DQ)

Under the Guidance of

Ms.HUMERA NAAZ

(Assistant Professor)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CMR TECHNICAL CAMPUS

UGC AUTONOMOUS

(Accredited by NAAC, NBA, Permanently Affiliated to JNTUH, Approved by AICTE, New Delhi)

Recognized Under Section 2(f) & 12(B) of the UGCAct.1956,

Kandlakoya (V), Medchal Road, Hyderabad-501401.

May-**2026.**

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the project entitled "**HIERARCHICAL STROKE-DENSITY PROFILING FOR OFFLINE AUTOGRAPH IDENTITY CONFIRMATION**" being submitted by **Malka Divya Sri (237R1A05DQ)** in partial fulfilment of the requirements for the award of the degree of B.Tech in Computer Science and Engineering to the Jawaharlal Nehru Technological University Hyderabad, during the year 2025-26.

The results embodied in this thesis have not been submitted to any other University or Institute for the award of any degree or diploma.

Ms.Humera Naaz
Assistant Professor
INTERNAL GUIDE

Dr. Nuthanakanti Bhaskar
HOD

Dr. A. Raji Reddy
DIRECTOR

Signature of External Examiner

Submitted for viva voice Examination held on _____

ACKNOWLEDGEMENT

I take this opportunity to express my gratitude to the people who have been instrumental in the successful completion of this project, I take this opportunity to express my profound gratitude and deep regard to my guide **Ms.Humera Naaz,** Assistant Professor for her exemplary guidance, monitoring and constant encouragement throughout the project work. The blessing, help and guidance given by her shall carry me a long way in the journey of life on which I am about to embark.

I take this opportunity to extend my heartfelt appreciation to the Project Review Committee (PRC) Coordinators—**Dr. Mantesh Patil, Mr.R. Sai Krishna, Ms. Tabeen Fatima, Mr.Deepakkumar,Ms.B.Pooja,Dr.PunyabanPatel,Ms.G.Swarnalatha,Mr.G.Vinesh Shankar**—for their unwavering support, insightful guidance, and valuable inputs, which played a crucial role in steering this project through its various stages.

My sincere appreciation also goes to **Dr. Nuthanakanti Bhaskar**, Head of the department , for his encouragement and continuous support in ensuring the successful completion of my project.

I am deeply grateful to **Dr. A. Raji Reddy**, Director, for his cooperation throughout the course of this project. Additionally, I extend my profound gratitude to Sri. **Ch. Gopal Reddy**, Chairman, Smt. **C. Vasantha Latha**, Secretary and Sri. **C. Abhinav Reddy**, Vice-Chairman, for fostering an excellent infrastructure and a conducive learning environment that greatly contributed to my progress.

I also acknowledge and appreciate the guidance and assistance provided by the faculty and staff of **CMR Technical Campus**, whose contributions have been invaluable in bringing this project to fruition.

Lastly, I sincerely thank my family for their unwavering support and encouragement. I also extend my gratitude to the teaching and non-teaching staff of CMR Technical Campus for their guidance and assistance. Their contributions, along with the support of everyone who helped directly or indirectly, have been invaluable in the successful completion of this project.

Malka Divya Sri (237R1A05DQ)

VISION AND MISSION

INSTITUTE VISION:

To Impart quality education in serene atmosphere thus strive for excellence in Technology and Research.

INSTITUTE MISSION:

1. To create state of art facilities for effective Teaching- Learning Process.
2. Pursue and Disseminate Knowledge based research to meet the needs of Industry & Society.
3. Infuse Professional, Ethical and Societal values among Learning Community.

DEPARTMENT VISION:

To provide quality education and a conducive learning environment in computer engineering that foster critical thinking, creativity, and practical problem-solving skills.

DEPARTMENT MISSION:

1. To educate the students in fundamental principles of computing and induce the skills needed to solve practical problems.
2. To provide State-of-the-art computing laboratory facilities to promote industry institute interaction to enhance student's practical knowledge.
3. To inculcate self-learning abilities, team spirit, and professional ethics among the students to serve society.

ABSTRACT

This project is titled as “Hierarchical Stroke-Density Profiling for Offline Autograph Identity Confirmation”. The increasing dependence on handwritten signatures for authentication in banking, legal, and administrative systems has led to a rise in signature forgery, creating a need for accurate and efficient verification techniques. Offline signature verification, in particular, is challenging due to the absence of dynamic writing information and the presence of skilled forgeries.To address these challenges, a novel hierarchical approach based on stroke-density profiling is proposed for signature verification and forgery detection. In this method, signature images are first preprocessed through binarization, resizing, and thinning to standardize the input data. The processed images are then divided into multiple hierarchical segments using structured partitioning techniques to capture both global and local stroke-density characteristics.Feature extraction is performed by analyzing stroke distribution, vector angles, and spatial distances within each segment, forming a robust representation of the signature. These features are further optimized and used for classification through machine learning and deep learning models such as Support Vector Machine (SVM), K-Nearest Neighbors (KNN), and Long Short-Term Memory (LSTM) networks. The dataset is split into training and testing sets to evaluate the system using performance metrics like accuracy, precision, recall, and F1-score.Experimental results indicate that the proposed hierarchical stroke-density profiling approach improves the accuracy and reliability of signature verification. Among the applied models, LSTM demonstrates superior performance due to its ability to capture sequential and structural patterns in signature data.Overall, the proposed system enhances the detection of forged signatures by effectively modeling both local and global signature features, making it suitable for real-world offline authentication systems.

LIST OF FIGURES

FIGURE NO	FIGURE NAME	PAGE NO
Figure 3.1	Project Architecture of A Hierarchical Stroke-Density Profiling for Offline Autograph Identity Confirmation	17
Figure 3.3	Dataflow Diagram of A Hierarchical Stroke -Density Profiling for Offline Autograph Identity Confirmation	20
Figure 4.1.1	Dataset directory structure with folders 'Forge' and 'Original' containing forged and genuine signature samples respectively	27
Figure 4.1.2	Screenshot of the "Forge" Folder Showing Sample Image	27
Figure 4.1.3	Screenshot of the "Original" Folder Showing sample images	28

Figure 5.1	GUI/Main Interface of A Hierarchical Stroke-Density Profiling for Offline Autograph Identity Confirmation	41
Figure 5.2	Loaded sample image of A Hierarchical Stroke-Density Profiling for Offline Autograph Identity Confirmation	42
Figure 5.3	Dataset of A Hierarchical Stroke-Density Profiling for Offline Autograph Identity Confirmation	43
Figure 5.4	Dataset Training and Testing Details of Hierarchical Stroke-Density Profiling for Offline Autograph Identity Confirmation	44
Figure 5.5	Feature Extraction and Selection of Hierarchical Stroke-Density Profiling for Offline Autograph Identity Confirmation	45
Figure 5.6	Evaluation Metrics and Confusion Matrix of the SVM Model for Hierarchical Stroke-Density Profiling for Offline Autograph Identity Confirmation	46

Figure 5.7	Evaluation Metrics and Confusion Matrix of the KNN Model for Heirarchical Stroke-Density Profiling for Offline Autograph Identity Confirmation	47
Figure 5.8	Evaluation Metrics and Confusion Matrix of the LSTM Model for Hierarchical Stroke-Density Profiling for Offline Autograph Identity Confirmation	48
Figure 5.9	Comparison graph of all three models of Hierarchical Stroke-Density Profiling for Offline Autograph Identity Confirmation	49
Figure 5.10	"Original" prediction result displayed in the system interface of Hierarchical Stroke-Density Profiling for Offline Autograph Identity Confirmation	50
Figure 5.11	"Forge" prediction result displayed in the system interface of Hierarchical Stroke-Density Profiling for Offline Autograph Identity Confirmation	51

LIST OF TABLES

TABLE NO	TABLE NAME	PAGE NO
Table 6.2.1	Uploading Dataset	54
Table 6.2.2	Classification	54

TABLE OF CONTENTS

ABSTRACT i
LIST OF FIGURES ii
LIST OF TABLES v
1. INTRODUCTION 2
1.1 PROJECT PURPOSE 2
1.2 PROJECT FEATURES 3
2. LITERATURE SURVEY 5
2.1 REVIEW OF RELATED WORK 9
2.2 DEFINITION OF PROBLEM STATEMENT 11
2.3 EXISTING SYSTEM 11
2.4 PROPOSED SYSTEM 13
2.5 OBJECTIVES 14
2.6 HARDWARE & SOFTWARE REQUIREMENTS 15
2.6.1 HARDWARE REQUIREMENTS 15
2.6.2 SOFTWARE REQUIREMENTS 15
3. SYSTEM ARCHITECTURE & DESIGN 17
3.1 PROJECT ARCHITECTURE 17
3.2 DESCRIPTION 18
3.3 DATA FLOW DIAGRAM 19
4. IMPLEMENTATION 22
4.1 ALGORITHMS USED 22
4.2 SAMPLE CODE 29
5. RESULTS & DISCUSSION 41
6. VALIDATION 53
6.1 INTRODUCTION 53
6.2 TEST CASES 54
6.2.1 UPLOADING DATASET 54
6.2.2 CLASSIFICATION 54
7. CONCLUSION & FUTURE ASPECTS 56
7.1 PROJECT CONCLUSION 56
7.2 FUTURE ASPECTS 57
8. BIBLIOGRAPHY 59
8.1 REFERENCES 59
8.2 GITHUB LINK 60

1.INTRODUCTION

1. INTRODUCTION

The project, titled "Hierarchical Stroke-Density Profiling for Offline Autograph Identity Confirmation," is designed to develop an advanced system capable of automatically verifying handwritten signatures and detecting forged ones. The system focuses on analyzing stroke patterns, density distribution, and structural characteristics of signatures to accurately differentiate between genuine and forged samples. It utilizes hierarchical segmentation and feature extraction techniques to capture both local and global signature features, ensuring reliable identity confirmation.

Given the extensive use of signatures in banking, legal, and administrative applications, manual verification is inefficient and prone to errors, especially with skilled forgeries. Therefore, an automated system is essential to improve accuracy and efficiency. This project employs image processing along with machine learning and deep learning models such as SVM, KNN, and LSTM to classify signatures effectively. The system is designed to handle large datasets with high accuracy and minimal processing time, making it suitable for real-world authentication systems and scalable deployment

1.1 PROJECT PURPOSE

The primary purpose of this project is to enhance the accuracy and efficiency of signature verification systems by automating the detection and classification of forged signatures. Traditional verification methods rely heavily on manual inspection, which is time-consuming and prone to errors, especially when dealing with skilled forgeries. By implementing advanced image processing and machine learning techniques, this project provides a more reliable, scalable, and accurate solution for distinguishing between genuine and forged signatures.

With the increasing use of signatures in financial, legal, and administrative transactions, there is a strong need for an automated system that can ensure secure authentication

By analyzing stroke patterns, density distribution, and structural features of signatures, the system reduces human effort, improves verification speed, and enhances decision accuracy. Moreover, it minimizes inconsistencies in manual verification and provides a robust framework for real-world applications such as banking security, document authentication, and identity verification systems.

1.2 PROJECT FEATURES

This project incorporates several key features to improve the accuracy and efficiency of signature verification and forgery detection:

Automated Signature Verification: The system utilizes a combination of Histogram of Oriented Gradients (HOG) and Convolutional Neural Networks (CNN) to extract meaningful features from signature images. These features capture both structural and spatial characteristics of signatures. Further processing using machine learning and deep learning models such as SVM, KNN, and LSTM enhances the system's ability to accurately distinguish between genuine and forged signatures.

Multi-Model Classification: Unlike traditional methods that rely on a single algorithm, this project implements multiple classification techniques including SVM, KNN, and LSTM. Each model is trained and evaluated using performance metrics such as accuracy, precision, recall, and F1-score. This comparative approach ensures better analysis, with LSTM demonstrating superior performance in capturing sequential and structural patterns in signatures.

Efficient Data Processing & Feature Selection: The system includes preprocessing steps such as normalization, shuffling, and dataset splitting to improve model performance. Additionally, a Decision Tree-based feature selection method is applied to identify the most relevant features from the extracted data, reducing complexity and improving classification efficiency.

By integrating these features, the proposed system provides an efficient, accurate, and automated solution for signature forgery detection, improving authentication reliability and reducing the risk of fraudulent activities.

2. LITERATURE SURVEY

2. LITERATURE SURVEY

With the increasing reliance on handwritten signatures for authentication in banking, legal, and administrative systems, signature verification has become a critical area of research. However, the presence of skilled forgeries and natural variations in genuine signatures makes accurate verification a challenging task. Traditional manual verification methods are time-consuming and prone to human error, necessitating the development of automated systems using advanced machine learning and deep learning techniques.

This literature survey reviews previous research in signature verification and forgery detection, focusing on various methodologies, their advantages, limitations, and areas for improvement. The discussion includes traditional machine learning approaches, deep learning models, hybrid architectures, feature extraction techniques, and optimization methods that contribute to improving verification accuracy.

Early research in offline signature verification primarily relied on traditional machine learning algorithms such as Support Vector Machines (SVM), K-Nearest Neighbors (KNN), and Decision Trees. These approaches used handcrafted features like pixel distribution, geometric properties, and texture patterns extracted from signature images. While these methods achieved moderate success, they were highly dependent on feature engineering and struggled with complex and skilled forgeries.

Vargas et al. (2007) introduced a Support Vector Machine (SVM)-based approach for offline signature verification, leveraging handcrafted features such as texture and shape descriptors to improve classification accuracy. Their method performed better than earlier statistical models by reducing misclassification rates but lacked the ability to capture complex structural variations in skilled forgeries. The emergence of deep learning brought significant improvements in signature verification. Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) became widely used for feature extraction and sequential learning, respectively.

Hafemann et al. (2017) demonstrated the effectiveness of CNNs in extracting spatial features from signature images. Their study compared different CNN architectures and showed improved accuracy over traditional machine learning models. However, CNNs struggled with capturing sequential stroke information, leading to misclassification in cases of highly similar forged signatures. Zhang et al. (2018) enhanced CNN-based verification by integrating deeper architectures for feature extraction. Their work achieved better accuracy, but the absence of temporal modeling limited its effectiveness in understanding structural dependencies within signatures.

To address the lack of sequential understanding in CNNs, researchers explored RNNs and Long Short-Term Memory (LSTM) networks. **Kumar et al. (2019)** trained an LSTM-based model on signature datasets, demonstrating that LSTMs could capture stroke sequence patterns effectively. However, their model faced challenges with longer and more complex signatures due to vanishing gradient issues, which affected performance.

Bidirectional LSTMs further improved upon standard LSTMs by processing signature data in both forward and backward directions, enhancing contextual understanding. Studies by Dutta et al. (2020) confirmed that BiLSTMs, when combined with CNN-based feature extractors, produced better classification results than standalone CNN or LSTM models. Recognizing the limitations of single-model approaches, researchers began integrating CNNs for spatial feature extraction with LSTM-based architectures for sequential learning. One such hybrid approach has shown superior performance in recent studies.

CNN models extract visual and structural features from signature images, while LSTM networks process these features sequentially to identify patterns of genuine and forged signatures. The addition of attention mechanisms further improves classification by focusing on the most relevant regions of a signature. Zhang et al. (2022) introduced an attention-based LSTM model for signature verification, demonstrating that attention layers significantly improved accuracy by reducing false positives and false negatives. These mechanisms are particularly useful in distinguishing subtle differences between genuine and skilled forged signatures.

Recent studies have highlighted the benefits of integrating ensemble learning methods with deep learning models. Singh et al. (2021) incorporated Random Forest classifiers with CNN-based feature extraction, demonstrating improved classification stability and reduced overfitting. The ensemble approach leveraged multiple predictions to refine final outputs, resulting in more reliable verification systems.

To further enhance performance, researchers have explored combining multiple feature extraction techniques. Methods such as Histogram of Oriented Gradients (HOG) have been integrated with CNN features to capture both edge-based and deep representations. This hybrid feature approach improves robustness against variations in writing style and noise in signature images.

Another emerging area of research is transfer learning, where pre-trained models are fine-tuned for signature verification tasks. Models trained on large datasets can be adapted to signature datasets with minimal training, reducing computational cost while maintaining high accuracy. This approach has proven effective in improving model generalization.

Additionally, data preprocessing techniques such as binarization, normalization, thinning, and segmentation have been widely used to improve input quality. Proper preprocessing ensures consistency and enhances the ability of models to learn discriminative features effectively.

Despite these advancements, several challenges remain in signature verification systems. Traditional machine learning models fail to capture complex patterns, while standalone CNNs lack sequential understanding. LSTM-based models, although effective, can be computationally expensive. The absence of optimized feature selection methods further limits efficiency.

To overcome these limitations, the proposed study integrates hierarchical stroke-density profiling with feature extraction techniques such as HOG and CNN, followed by classification using SVM, KNN, and LSTM. This hybrid approach enhances spatial feature extraction, sequential learning, and classification accuracy, providing a reliable framework for signature verification.

Additionally, recent research has explored the integration of explainable AI (XAI) techniques in signature verification systems. XAI methods such as Grad-CAM and SHAP provide interpretability to deep learning models, enabling analysts to understand the reasoning behind classification decisions. This transparency is important for building trust in automated authentication systems, ensuring fairness, and reducing bias in signature verification models. Studies have shown that combining XAI with deep learning architectures improves reliability and acceptance in real-world applications.

Another emerging trend in signature verification research is the use of data augmentation techniques. Due to limited availability of large-scale signature datasets and privacy concerns, researchers have applied augmentation methods such as rotation, scaling, translation, and noise addition to generate diverse training samples. In some cases, Generative Adversarial Networks (GANs) have been used to create synthetic signature data. These techniques enhance model generalization and robustness by exposing classifiers to a wide range of variations, improving performance on real-world signature datasets.

Most prior research has focused on distinguishing between genuine and forged signatures based on static image features. However, challenges such as skilled forgeries and intra-class variations remain significant. Ethical concerns and dataset limitations have made it difficult to obtain diverse and high-quality signature samples. To address this, researchers have developed curated datasets and standardized evaluation protocols to ensure reliable and fair model training.

Despite advancements in signature verification, several limitations still exist. Traditional machine learning models lack the ability to capture complex structural patterns. CNN-based models focus mainly on spatial features and fail to capture sequential dependencies. LSTM-based models, although effective in sequence learning, suffer from higher computational complexity and longer training times. Additionally, the absence of optimized feature selection techniques can reduce overall system efficiency.

To overcome these limitations, the proposed study introduces a hierarchical stroke-density profiling approach combined with feature extraction methods such as HOG and CNN, along with classification models including SVM, KNN, and LSTM.

This hybrid framework enhances spatial feature extraction, sequential learning, and classification accuracy, providing a robust and reliable solution for signature verification.

The literature survey highlights the evolution of signature verification techniques from traditional machine learning models to advanced deep learning architectures. By leveraging CNNs, LSTMs, feature selection methods, and hybrid approaches, researchers have significantly improved verification accuracy and system reliability. The proposed system builds upon these advancements to achieve better performance and efficiency.The survey also emphasizes the growing need for automated and scalable signature verification systems. Traditional approaches struggle to handle large datasets and complex variations in signatures. Recent techniques have shown promising improvements, but further research is required to enhance real-time verification, optimize computational efficiency, and expand datasets for better generalization in real-world applications.

2.1 REVIEW OF RELATED WORK

The detection and verification of handwritten signatures have been extensively studied in the fields of pattern recognition, image processing, and machine learning. Several approaches have been proposed to address the problem, ranging from manual verification techniques to advanced deep learning-based models. This review discusses previous research and existing methodologies, highlighting their strengths and limitations.

1. Traditional Signature Verification Approaches

Early methods of signature verification relied on manual inspection and rule-based techniques. These approaches involved visual comparison of signatures by experts, focusing on features such as shape, size, and writing style. While effective for simple verification tasks, these methods were time-consuming, subjective, and prone to human error. Additionally, they lacked scalability and consistency, making them unsuitable for large-scale applications.

2. Machine Learning-Based Approaches

With the advancement of machine learning, researchers explored automated signature verification using classifiers such as Support Vector Machines (SVM), K-Nearest Neighbors (KNN), Decision Trees, and Random Forest. These methods relied on handcrafted features such as geometric properties, pixel distribution, texture patterns, and Histogram of Oriented Gradients (HOG). While these approaches improved accuracy compared to traditional methods, they were highly dependent on feature engineering and struggled with complex variations in signatures, especially in the case of skilled forgeries.

3. Deep Learning-Based Approaches

Recent developments in deep learning have significantly improved the performance of signature verification systems. Convolutional Neural Networks (CNNs) have been widely used for extracting high-level spatial features from signature images. For capturing sequential and structural relationships, models such as Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks have been employed.

One of the most effective approaches is the hybrid CNN-LSTM model, where CNNs extract spatial features and LSTMs analyze sequential dependencies. Studies have shown that pre-trained CNN models can effectively capture complex patterns in signatures, while LSTM layers improve classification by learning structural relationships. However, these models often require large datasets and involve high computational costs, which can limit real-time implementation.

4. Recent Advances: Feature Selection & Hybrid Models

To improve performance, recent research has focused on combining multiple techniques such as feature extraction, feature selection, and hybrid classification models. Decision Tree-based feature selection methods are used to identify the most relevant features, reducing dimensionality and improving efficiency. Additionally, combining HOG with CNN features enhances robustness by capturing both edge-based and deep representations.Hybrid models integrating CNN, LSTM, and traditional classifiers have shown promising results in improving accuracy and reducing false predictions.

5. Comparison with the Proposed Approach

While existing methods have made significant progress in signature verification, challenges remain in terms of accuracy, computational efficiency, and handling skilled forgeries. The proposed approach in this project builds upon previous work by introducing hierarchical stroke-density profiling combined with feature extraction techniques such as HOG and CNN, along with classification models like SVM, KNN, and LSTM.

This review highlights the evolution of signature verification techniques, emphasizing the transition from manual and machine learning approaches to advanced deep learning-based systems. The proposed methodology aims to address the limitations of previous research by providing a more accurate, efficient, and scalable solution for offline signature verification

2.2 DEFINITION OF PROBLEM STATEMENT

The primary goal of this project is to develop a robust and scalable system capable of accurately verifying handwritten signatures and detecting forged ones using advanced machine learning and deep learning techniques. This involves analyzing stroke patterns, density distribution, and structural features of signatures, effectively distinguishing between genuine and forged samples, handling variations in writing styles, and improving classification accuracy. By addressing these challenges, the system aims to enhance authentication reliability, reduce manual verification effort, and provide a secure and efficient solution for real-world applications such as banking, legal documentation, and identity verification systems.

2.3 EXISTING SYSTEM

The existing systems for offline signature verification utilize a combination of image processing techniques and machine learning classifiers to distinguish between genuine and forged signatures. One common approach involves extracting handcrafted features such as geometric properties, pixel intensity distributions, texture patterns, and Histogram of Oriented Gradients (HOG).

These features are then classified using algorithms like Support Vector Machines (SVM) with Radial Basis Function (RBF) kernels, K-Nearest Neighbors (KNN), and Decision Trees. Additionally, some systems incorporate zoning techniques, where signature images are divided into multiple regions to capture local characteristics. Feature descriptors from each region are processed individually and later combined to improve classification accuracy. Several studies have also explored the use of statistical and structural features, including stroke width, curvature, and spatial relationships between signature components. Hidden Markov Models (HMM) and Neural Networks have been applied to model sequential patterns in signatures. More advanced systems integrate Convolutional Neural Networks (CNNs) for automatic feature extraction, eliminating the need for manual feature engineering. Hybrid models combining CNNs with traditional classifiers have shown improved performance. Furthermore, preprocessing techniques such as binarization, thinning, normalization, and noise removal are widely used to enhance input quality. Collectively, these methods provide a comprehensive framework for signature verification by combining feature extraction and classification techniques.

Limitations of Existing System

Despite advancements in signature verification, the existing systems suffer from the following challenges:

- Inability to capture complex structural relationships: Traditional machine learning models rely on handcrafted features and fail to capture intricate variations in signature patterns. CNNs focus mainly on spatial features but lack the ability to model sequential stroke information effectively.

- High false positive and false negative rates: Many systems struggle to differentiate between genuine signatures and skilled forgeries. Variations in writing style, pressure, and speed can lead to incorrect classification, reducing system reliability.

- Dependence on preprocessing and feature selection: Many existing approaches rely heavily on preprocessing techniques and manual feature extraction, which can affect performance if not properly optimized.

- Computational inefficiency: Deep learning models, especially CNN-based systems, require significant computational resources and large datasets for training. Processing high-resolution signature images can increase time and memory requirements.

- Overfitting and lack of generalization: Models trained on limited datasets may not perform well on unseen signature samples. Differences in writing styles across individuals can reduce the generalization capability of the system.

2.4 PROPOSED SYSTEM

The proposed system introduces a novel hybrid framework for offline signature verification using hierarchical stroke-density profiling combined with machine learning and deep learning techniques. A dataset of handwritten signature images, consisting of both genuine and forged samples, is used to train and evaluate the system. The input signatures are preprocessed through binarization, resizing, normalization, and thinning to ensure consistency and improve feature quality. The processed images are then divided into hierarchical segments to effectively capture stroke density, spatial distribution, and structural characteristics of signatures. Feature extraction is performed using Histogram of Oriented Gradients (HOG) along with Convolutional Neural Networks (CNN) to obtain both edge-based and deep features. These features are further optimized using a Decision Tree-based feature selection technique to retain only the most relevant attributes. The refined feature set is then used to train multiple classification models such as Support Vector Machine (SVM), K-Nearest Neighbors (KNN), and Long Short-Term Memory (LSTM) networks to accurately classify signatures as genuine or forged. The system is evaluated using performance metrics including accuracy, precision, recall, and F1-score, where experimental results indicate that deep learning models, particularly LSTM, achieve superior performance compared to traditional methods. Overall, the proposed approach demonstrates high accuracy, reduced false predictions, and improved robustness

Advantages of the Proposed System:

The proposed system significantly improves upon the existing approaches by addressing key limitations:

- Higher Accuracy: The combination of hierarchical stroke-density profiling with HOG, CNN, and LSTM provides a comprehensive feature representation of signatures. This reduces false acceptance and false rejection rates, making the verification process more reliable.

- Improved Structural Understanding: The use of hierarchical segmentation and LSTM enables the system to capture both local and global structural patterns of signatures, improving its ability to distinguish between genuine and skilled forged signatures.

- Better Generalization & Robustness: Decision Tree-based feature selection helps in selecting the most relevant features, reducing overfitting and improving the model's ability to perform well on unseen signature data.

- Efficient Processing: The integration of optimized feature extraction techniques such as HOG and CNN reduces computational complexity while maintaining high accuracy. Compared to traditional methods, the proposed system is faster and more suitable for large-scale applications.

- Enhanced Interpretability & consistency: The use of multiple classifiers such as SVM, KNN, and LSTM ensures stable and consistent performance, minimizing errors caused by variations in writing style and signature patterns.

2.5 OBJECTIVES

- Develop an Automated Signature Verification System – Build a machine learning and deep learning-based model to automatically verify handwritten signatures and detect forged ones.

- Enhance Automation Efficiency – Reduce dependence on manual verification by implementing an AI-based system for faster and more reliable signature validation.
- Improve Detection Accuracy – Utilize hierarchical stroke-density profiling along with HOG, CNN, and LSTM models to achieve higher accuracy compared to traditional methods.
- Ensure Scalability – Develop the system to handle large volumes of signature

data efficiently, making it suitable for real-world applications such as banking and document authentication.

- Ensure Robustness Against Forgeries – Design the system to effectively distinguish between genuine and skilled forged signatures by capturing both local and global structural features.

2.6 HARDWARE & SOFTWARE REQUIREMENTS

2.6.1 HARDWARE REQUIREMENTS:

Hardware interfaces specifies the logical characteristics of each interface between the software product and the hardware components of the system. The following are some hardware requirements,

- Processor : Intel Core i3
- Hard disk : 20GB.
- RAM : 4GB.

2.6.2 SOFTWARE REQUIREMENTS:

Software Requirements specifies the logical characteristics of each interface and software components of the system. The following are some software requirements,

- Operating system : Windows 10
- Language : Python
- Libraries/Tools : NumPy, Pandas, OpenCV, Scikit-learn, TensorFlow/Keras
- IDE/Environment : Jupyter Notebook/VS Code
- Frame Work : Tkinter

3.SYSTEM ARCHITECTURE & DESIGN

3. SYSTEM ARCHITECTURE & DESIGN

Project architecture refers to the structural framework and design of a project, encompassing its components, interactions, and overall organization. It provides a clear blueprint for development, ensuring efficiency, scalability, and alignment with project goals. Effective architecture guides the project's lifecycle, from planning to execution, enhancing collaboration and reducing complexity.

3.1 PROJECT ARCHITECTURE

This project architecture shows the procedure followed in Hierarchical Stroke-Density Profiling for Offline Autograph Identity Confirmation, starting from input signature images to final classification as genuine or forged.

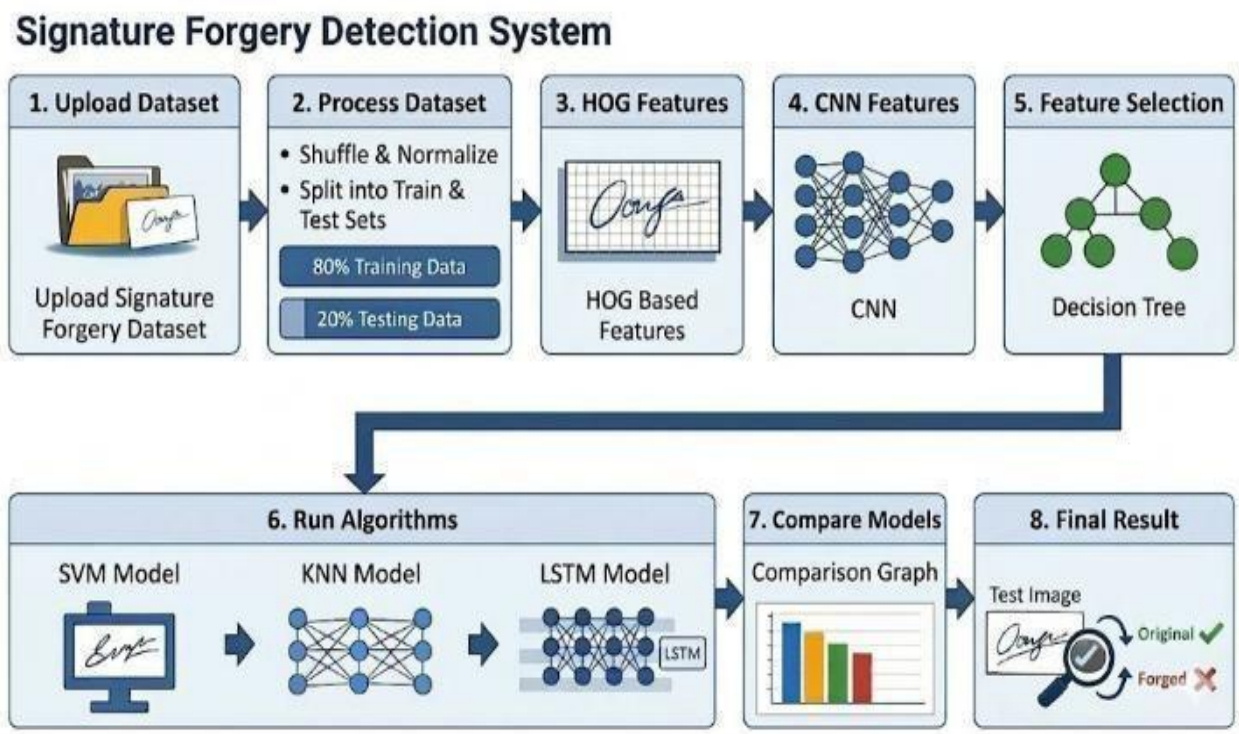

Figure 3.1: Project Architecture of A Hierarchical Stroke-Density Profiling for Offline Autograph Identity Confirmation

3.2 DESCRIPTION

Input Data : The project uses a dataset of genuine and forged handwritten signatures for supervised learning.

Reading Data: The dataset is shuffled, normalized, and split into 80% training and 20% testing sets.

Feature Extraction : HOG-based features and CNN-based features are extracted from signature images.

Feature Selection: A decision tree is applied to select the most relevant features for classification

Model Training : SVM, KNN, and LSTM models are trained on the selected features.

Model Comparison: The performance of the models is compared using evaluation metrics and graphs.

Final Classification : The best-performing model classifies signatures as original or forged.

3.3 DATA FLOW DIAGRAM

The Data Flow Diagram (DFD) of the Signature Forgery Verification System describes how data moves through different stages of the system and how it is transformed to produce the final result. It provides a clear understanding of the internal working of the system from input to output. The process begins with the user, who uploads the signature dataset into the system. This dataset contains both genuine and forged signatures, which act as input for the system. Once the dataset is uploaded, it is stored temporarily and passed to the next stage for further processing. This step is important because the quality and size of the dataset directly affect the performance of the system.

After the dataset is uploaded, the system performs data processing. In this stage, the raw data is prepared for analysis. The dataset is first shuffled to remove any bias in the data order. Then normalization is applied to scale the data into a uniform format, which helps in improving the performance of machine learning algorithms. After normalization, the dataset is divided into training and testing sets using the train-test split method. The training data is used to train the models, while the testing data is used to evaluate their performance.

Once the data processing is completed, the system moves to the feature extraction stage. This is one of the most important steps in the entire system. In this stage, meaningful features are extracted from the signature images using techniques such as Histogram of Oriented Gradients (HOG) and Convolutional Neural Networks (CNN). HOG helps in capturing the edge and shape information of the signatures, while CNN automatically learns complex patterns from the images. These extracted features represent the unique characteristics of each signature.

After extracting features, the system performs feature selection. In this stage, unnecessary or less important features are removed to improve the efficiency of the system. The Decision Tree algorithm is used to select the most relevant features. This reduces the dimensionality of the data and helps in faster and more accurate classification.

The selected features are then passed to the classification stage. In this stage, multiple machine learning models such as Support Vector Machine (SVM), K-Nearest Neighbors (KNN), and Long Short-Term Memory (LSTM) are used. Each model analyzes the features and classifies the signatures based on learned patterns. SVM separates data using optimal boundaries, KNN classifies based on similarity with nearest data points, and LSTM captures sequential patterns in the data.

After classification, the system performs model comparison. The performance of each algorithm is evaluated using metrics such as accuracy, precision, recall, and F1-score. The results are displayed in the form of an accuracy graph. This helps in identifying which model performs better for the given dataset. In most cases, advanced models like LSTM tend to provide better accuracy.

Finally, the system produces the predicted result. Based on the analysis, the system determines whether the given signature is genuine or forged. The result is then displayed to the user in a clear and understandable format.

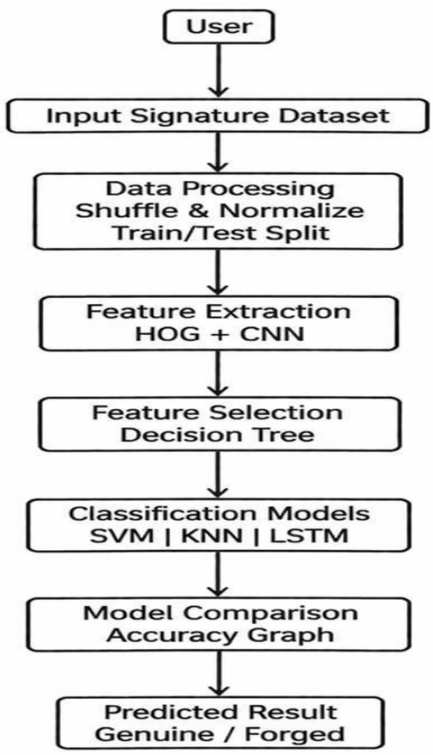

Figure 3.3: Dataflow Diagram of A Hierarchical Stroke-Density Profiling for Offline Autograph Identity Confirmation

4.IMPLEMENTATION

4. IMPLEMENTATION

The implementation phase of the project involves executing the planned methodology for signature verification and forgery detection. It includes data collection, preprocessing, feature extraction, feature selection, model training, and evaluation. Proper coordination of these steps ensures that the system achieves high accuracy and reliability in distinguishing between genuine and forged signatures. Efficient implementation is essential for delivering accurate results within limited computational resources and time constraints.

4.1 ALGORITHMS USED

CNN-Based Models for Feature Extraction

Convolutional Neural Networks (CNNs) are widely used in this project for extracting deep features from signature images. CNNs automatically learn important spatial patterns such as curves, edges, and stroke structures present in handwritten signatures. By identifying these patterns, CNNs help in capturing complex characteristics that differentiate genuine signatures from forged ones.

Advantages of CNN-Based Models:

• Highly effective in extracting complex visual and structural patterns from signature images.
• Eliminates the need for manual feature engineering.
• Provides high accuracy in image-based classification tasks

Disadvantages of CNN-Based Models:

- Requires large datasets for effective training.
- Computationally intensive, especially for high-resolution images.
- Focuses mainly on spatial features and may miss sequential relationships.

LSTM-Based Models for Structural Analysis

To enhance the verification process, Long Short-Term Memory (LSTM) networks are used to capture structural relationships in signatures. While CNN extracts spatial features, LSTM processes feature sequences to understand stroke flow, writing order, and structural consistency. This helps in identifying subtle differences between genuine signatures and skilled forgeries. LSTM models are particularly useful in modeling dependencies within extracted features, which improves classification accuracy.

Advantages of LSTM Models:

- Effective in capturing structural and sequential relationships in signatures.
- Improves classification accuracy by learning complex dependencies.
- Handles variations in writing styles and patterns efficiently.
- Suitable for detecting skilled and complex forgeries.

Disadvantages of LSTM Models:

- Computationally intensive and require more training time.
- Performance depends heavily on quality of extracted features.
- Requires careful tuning of parameters.
- May increase overall system complexity.

HOG (Histogram of Oriented Gradients) for Feature Extraction

- Captures stroke direction, edge orientation, and shape details effectively.
- Provides a strong representation of signature structure.
- Works efficiently with machine learning classifiers.
- Enhances overall feature quality when combined with CNN features.

CNN for Optimized Feature Extraction

CNN plays a key role in extracting high-level features from signature images. It uses multiple convolutional layers to automatically learn patterns such as curves, edges, and complex structures present in signatures.

- Automatically learns important spatial features.
- Identifies complex and hidden patterns in signatures.
- Reduces dependency on manual feature extraction.
- Improves system performance when integrated with other models.

LSTM for Structural Awareness

- Processes feature sequences to understand stroke continuity.
- Helps in distinguishing genuine signatures from forged ones.
- Enhances accuracy by learning dependencies in extracted features.
- Improves robustness of the system against skilled forgeries.

Decision Tree for Feature Selection

Decision Tree is used to select the most relevant features from extracted data by analyzing their importance in classification. It works by splitting the data based on feature values and identifying which features contribute most to correct predictions. By removing less important features, the system becomes more efficient and faster, while also reducing the chances of overfitting.

Why Use Decision Tree?

- Reduces dimensionality by selecting important features.
- Improves efficiency and reduces computational cost.
- Helps prevent overfitting and improves generalization.
- Simplifies the model by removing irrelevant features.

How Does Decision Tree Fit in the System?

- Features extracted from HOG and CNN are passed to the Decision Tree for feature selection.
- The Decision Tree identifies and selects the most important features based on their contribution to classification.
- Helps reduce irrelevant and redundant data, improving model efficiency and accuracy.
- Helps reduce irrelevant and redundant data, improving model efficiency and accuracy.
- Acts as a preprocessing step before applying classifiers like SVM, KNN, and LSTM.
- Improves generalization and reduces overfitting by focusing only on significant features.

Support Vector Machine(SVM):

In this project, SVM is used as a baseline classifier for signature verification. It works by finding the optimal boundary (hyperplane) that separates genuine and forged signatures based on selected features. SVM is effective for classification tasks where the data is well-structured and separable. However, it has limitations when dealing with complex and highly non-linear patterns in signature. Support Vector Machine (SVM) is used in this project as a baseline classifier to classify signatures as genuine or forged based on the selected features. It provides good accuracy when working with structured feature data and is particularly effective for small to medium-sized datasets. However, SVM has limitations when handling complex and non-linear patterns in signature data, which makes it less effective compared to advanced deep learning models

Now-a-days, handwritten signatures are widely used for identity verification in banking, legal documents, and official transactions. With the increasing use of digital systems, there is also a rise in signature forgery, where malicious users attempt to imitate genuine signatures for unauthorized access or fraud. Traditional manual verification methods are time-consuming and prone to human error, making it necessary to develop an automated system for accurate and efficient signature authentication.

To address this issue, this project employs a combination of image processing and machine learning techniques to detect forged signatures. Initially, signature images are processed using HOG (Histogram of Oriented Gradients) to extract edge and stroke-based features. These features are further enhanced using a Convolutional Neural Network (CNN), which captures complex spatial patterns present in signatures. To improve structural understanding, LSTM is used to analyze feature sequences and identify subtle differences between genuine and forged signatures, thereby enhancing prediction accuracy.

In the proposed work, a hybrid architecture is developed for offline autograph identity confirmation. The system integrates HOG and CNN for feature extraction, followed by Decision Tree for feature selection to identify the most relevant attributes. The selected features are then used to train multiple classifiers such as SVM, KNN, and LSTM. Among these, LSTM provides better performance due to its ability to capture complex relationships in the data. A comparison of all algorithms is performed to evaluate their accuracy and efficiency.

Since collecting large-scale real-world forged signature datasets is challenging, the system is trained and tested on available signature datasets containing both genuine and forged samples. The proposed approach focuses on distinguishing normal (genuine) signatures from forged ones, ensuring reliable authentication. Experimental results show that the hybrid model improves accuracy compared to traditional methods like SVM and KNN. This system can be effectively used in real-world applications such as banking security, document verification, and identity authentication to prevent fraud and ensure data integrity.

In the proposed work, collecting real-world forged signature datasets is difficult due to privacy and security concerns, Therefore, we have used a dataset consisting of “Genuine” and “Forged” signature images to train the models. In this context, genuine signatures are considered as valid samples (Figure 4.2) and forged signatures are treated as fraudulent samples (Figure 4.3).

To train all algorithms, we have used a signature dataset organized into two folders (Figure 4.1) named "Genuine" and "Forged," and the below screen shows the dataset details.

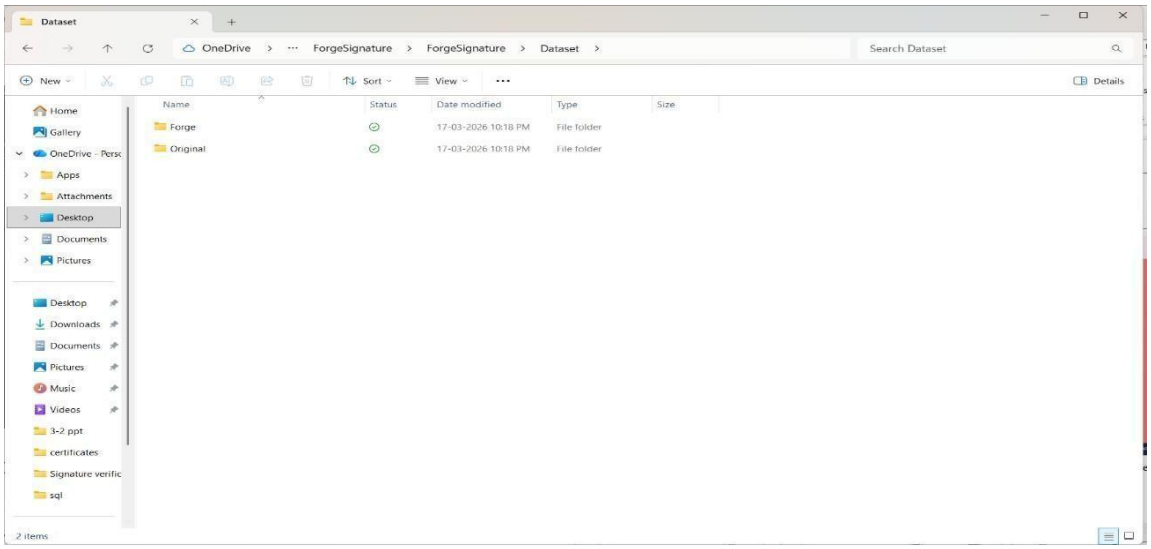

Figure 4.1.1: Dataset directory structure with folders 'Forge' and 'Original' containing forged and genuine signature samples respectively

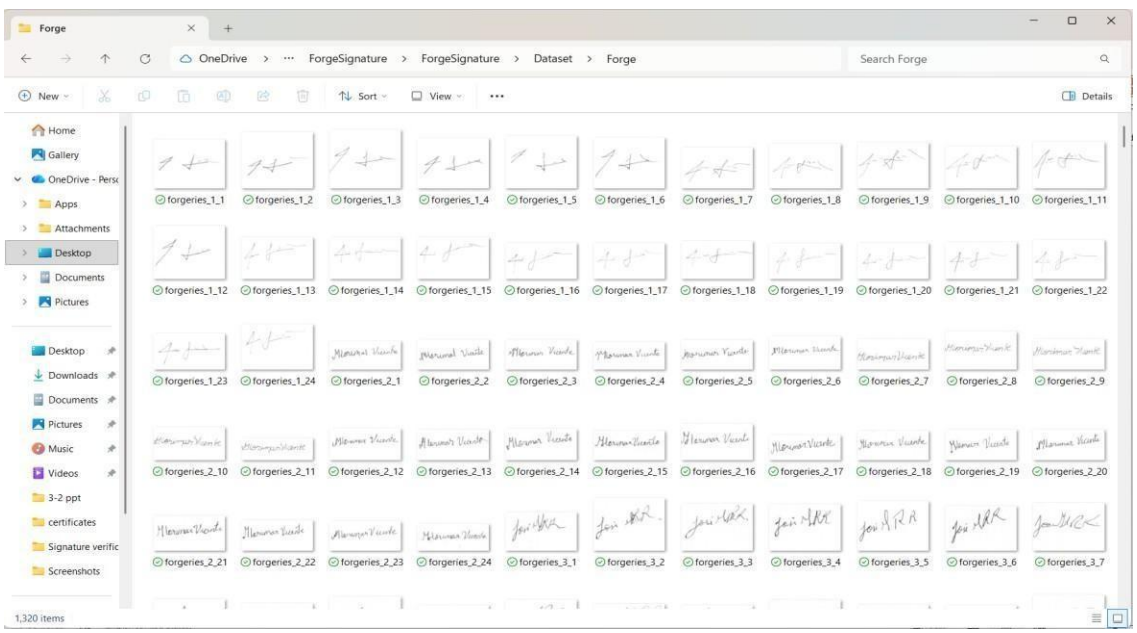

Figure 4.1.2: Screenshot of the "Forge" Folder Showing Sample Image

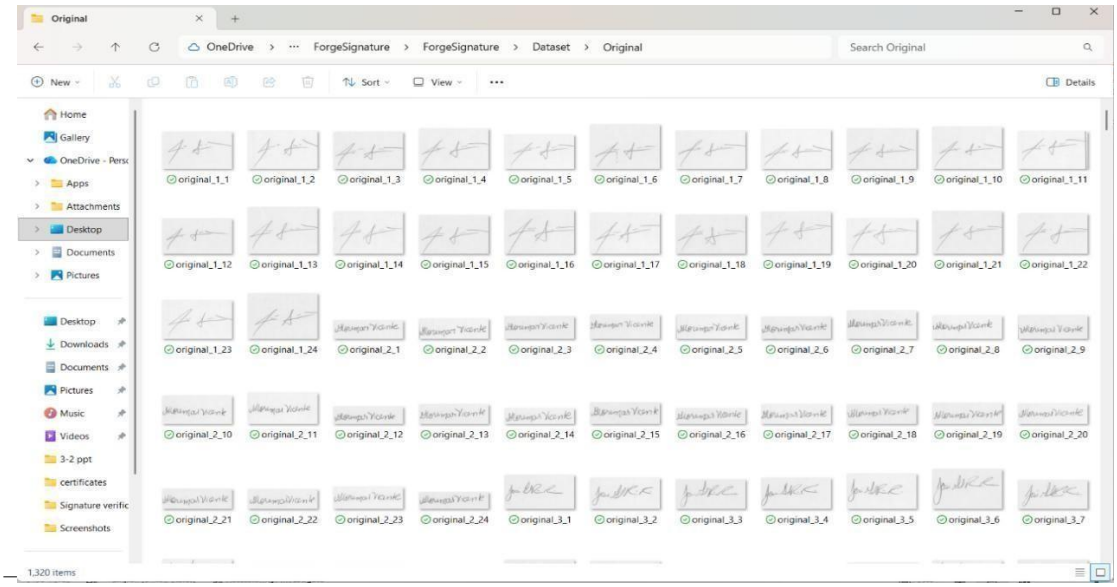

Figure 4.1.3: Screenshot of the " Original " Folder Showing Sample Image.

To implement this project we have designed following modules:

1) **Upload Signature Dataset:** using this module we will upload signature images dataset containing both *Original* and *Forged* signatures to the application (Figure 4.1).
2) **Dataset Preprocessing:** using this module we will read all signature images, resize them into a uniform size, convert them into grayscale/binary format, normalize pixel values, and shuffle the dataset for better training performance.
3) **Run Proposed CNN–HOG–LSTM Algorithm:** using this module we will split the dataset into training and testing sets. Then 80% of training data is given to CNN (Convolutional Neural Network) and HOG (Histogram of Oriented Gradients) for feature extraction, followed by feature selection using Decision Tree. The selected features are then trained using LSTM algorithm to build the model. Trained model will be applied on 20% test data to calculate prediction accuracy.
4) **Run CNN–HOG–SVM Algorithm:** CNN and HOG extracted features will be retrained using SVM (Support Vector Machine) algorithm and then calculate prediction accuracy.
5) **Run CNN–HOG–KNN Algorithm:** CNN and HOG extracted features will be retrained using KNN (K-Nearest Neighbors) algorithm and then calculate prediction accuracy.
6) **Comparison Graph:** using this module we will plot accuracy comparison graph between CNN–HOG–LSTM, CNN–HOG–SVM and CNN–HOG–KNN models.
7) **Signature Forgery Prediction from Test Image:** using this module we will upload any test signature image and the system will predict .

4.2 SAMPLE CODE

Main.py

```
from tkinter import *
import tkinter
from tkinter import filedialog
import numpy as np
from tkinter import simpledialog
from tkinter import ttk
from tkinter.filedialog import askopenfilename
import cv2
import os
import numpy as np
#loading python require packages
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from keras.models import Sequential, load_model, Model
import pickle
import seaborn as sns
from sklearn.metrics import accuracy_score
from keras.layers import Dense, Flatten, Dropout, Conv3D, MaxPooling3D, LSTM,RepeatVector
from keras.utils import to_categorical
from sklearn.metrics import roc_curve
from sklearn.metrics import roc_auc_score
from sklearn import metrics
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
from sklearn.metrics import confusion_matrix
```

```
from skimage import feature
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn import svm
from keras.callbacks import ModelCheckpoint
from keras.layers import  MaxPooling2D
from keras.layers import Dense, Dropout, Activation, Flatten, GlobalAveragePooling2D,
BatchNormalization, AveragePooling2D
from keras.layers import Convolution2D


main = tkinter.Tk()
main.title("Signature Forgery Verification") #designing main screen
main.geometry("1000x650")


global filename, X, Y
global X_train, X_test, y_train, y_test, cnn_model
global accuracy, precision, recall, fscore

def loadDataset():
    global X, Y
    if os.path.exists("model/X.txt.npy"):
        X = np.load('model/X.txt.npy')
        Y = np.load('model/Y.txt.npy')
    else:
        for root, dirs, directory in os.walk(filename):
            for j in range(len(directory)):
                name = os.path.basename(root)
                if 'Thumbs.db' not in directory[j]:
                    img = cv2.imread(root+"/"+directory[j])
                    img = cv2.resize(img, (32, 32))
```

```
            img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
            hog_features, img = feature.hog(img, orientations=9, pixels_per_cell=(8, 8),
cells_per_block=(2, 2), visualize=True, transform_sqrt=True, block_norm='L1')
            X.append(hog_features)
            label = 0
            if name == 'Forge':
                label = 1
            Y.append(label)
    X = np.asarray(X)
    Y = np.asarray(Y)
    np.save("model/X.txt.npy",X)
    np.save("model/Y.txt.npy",Y)

def uploadDataset():
  global filename, X, Y
  filename = filedialog.askdirectory(initialdir=".")
  text.delete('1.0', END)
  text.insert(END,filename+" loaded\n\n")
  X = []
  Y = []
  loadDataset()
  text.insert(END,"Image Processing & HOG Features extracted Completed\n\n")
  text.insert(END,"Total images loaded = "+str(X.shape[0])+"\n\n")
  text.insert(END,"Number of HOG features extracted from each image = "+str(X.shape[1]))

def processDataset():
  global X, Y
  global X_train, X_test, y_train, y_test
  text.delete('1.0', END)
  indices = np.arange(X.shape[0])
  np.random.shuffle(indices)
  X = X[indices]
```

```
    Y = Y[indices]
    Y = to_categorical(Y)
    X = np.reshape(X, (X.shape[0], X.shape[1], 1, 1))
    X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.2) #split dataset into train
and test
    text.insert(END,"Image Shuffling & Normalization Completed\n\n")
    text.insert(END,"Dataset Training & Testing Details\n\n")
    text.insert(END,"80% images for training : "+str(X_train.shape[0])+"\n")
    text.insert(END,"20% images for testing  : "+str(X_test.shape[0])+"\n")

#function to calculate all metrics
def calculateMetrics(algorithm, testY, predict):
    p = precision_score(testY, predict,average='macro') * 100
    r = recall_score(testY, predict,average='macro') * 100
    f = f1_score(testY, predict,average='macro') * 100
    a = accuracy_score(testY,predict)*100
    accuracy.append(a)
    precision.append(p)
    recall.append(r)
    fscore.append(f)
    text.insert(END,algorithm+" Accuracy  : "+str(a)+"\n")
    text.insert(END,algorithm+" Precision : "+str(p)+"\n")
    text.insert(END,algorithm+" Recall    : "+str(r)+"\n")
    text.insert(END,algorithm+" FSCORE    : "+str(f)+"\n\n")
    labels = ['Original', 'Forge']
    conf_matrix = confusion_matrix(testY, predict)
    fig, axs = plt.subplots(1,2,figsize=(10, 3))
    ax = sns.heatmap(conf_matrix, xticklabels = labels, yticklabels = labels, annot = True,
cmap="viridis" ,fmt ="g", ax=axs[0]);
    ax.set_ylim([0,len(labels)])
    axs[0].set_title(algorithm+" Confusion matrix")
```

```
    random_probs = [0 for i in range(len(testY))]
    p_fpr, p_tpr, _ = roc_curve(testY, random_probs, pos_label=1)
    plt.plot(p_fpr, p_tpr, linestyle='--', color='orange',label="True classes")
    ns_tpr, ns_fpr, _ = roc_curve(testY, predict, pos_label=1)
    axs[1].plot(ns_tpr, ns_fpr, linestyle='--', label='Predicted Classes')
    axs[1].set_title(algorithm+" ROC AUC Curve")
    axs[1].set_xlabel('False Positive Rate')
    axs[1].set_ylabel('True Positive rate')
    plt.show()

def runCNN():
    global X_train, X_test, y_train, y_test, cnn_model, X, Y
    text.delete('1.0', END)
    cnn_model = Sequential()
    cnn_model.add(Convolution2D(32, (1 , 1), input_shape = (X_train.shape[1], X_train.shape[2],
X_train.shape[3]), activation = 'relu'))
    cnn_model.add(MaxPooling2D(pool_size = (1, 1)))
    cnn_model.add(Convolution2D(32, (1, 1), activation = 'relu'))
    cnn_model.add(MaxPooling2D(pool_size = (1, 1)))
    cnn_model.add(Flatten())
    cnn_model.add(Dense(units = 256, activation = 'relu'))
    cnn_model.add(Dense(units = y_train.shape[1], activation = 'softmax'))
    cnn_model.compile(optimizer = 'adam', loss = 'categorical_crossentropy', metrics =
['accuracy'])
    cnn_model.compile(optimizer = 'adam', loss = 'categorical_crossentropy', metrics =
['accuracy'])
    if os.path.exists("model/cnn_weights.hdf5") == False:
        model_check_point = ModelCheckpoint(filepath='model/cnn_weights.hdf5', verbose = 1,
save_best_only = True)
        hist = cnn_model.fit(X_train, y_train, batch_size = 16, epochs = 20,
validation_data=(X_test, y_test), callbacks=[model_check_point], verbose=1)
        f = open('model/cnn_history.pckl', 'wb')
```

```
        pickle.dump(hist.history, f)
        f.close()
    else:
        cnn_model.load_weights("model/cnn_weights.hdf5")
    print(cnn_model.summary())
    cnnmodel = Model(cnn_model.inputs, cnn_model.layers[-2].output)#create mobilenet model
    X = cnnmodel.predict(X)  #extracting mobilenet features
    Y = np.argmax(Y, axis=1)
    text.insert(END,"CNN & HOG Based Features Extraction Completed\n\n")
    text.insert(END,"Total features extracted from each image is : "+str(X.shape[1])+"\n\n")

def runDT():
    global X, Y
    dt = DecisionTreeClassifier()
    dt.fit(X, Y)
    importances = dt.feature_importances_
    threshold = 0.0005
    selected_features = [i for i, importance in enumerate(importances) if importance > threshold]
    selected_features = np.asarray(selected_features)
    text.insert(END,"Decision Tree Based Features Selection Completed\n")
    text.insert(END,"Total features Selected by Decision Tree = "+str(selected_features.shape[0])+"\n\n")

def runSVM():
    text.delete('1.0', END)
    global X, Y
    global accuracy, precision, recall, fscore, X_train, X_test, y_train, y_test
    accuracy = []
    precision = []
    recall = []
    fscore = []
    X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.2) #split dataset into train
```

```
and test
    svm_cls = svm.SVC()
    svm_cls.fit(X_train, y_train)
    predict = svm_cls.predict(X_test)
    calculateMetrics("SVM", y_test, predict)

def runKNN():
    global accuracy, precision, recall, fscore, X_train, X_test, y_train, y_test
    knn_cls = KNeighborsClassifier(n_neighbors=2)
    knn_cls.fit(X_train, y_train)
    predict = knn_cls.predict(X_test)
    calculateMetrics("KNN", y_test, predict)

def runLSTM():
    global accuracy, precision, recall, fscore, X_train, X_test, y_train, y_test
    X_train1 = np.reshape(X_train, (X_train.shape[0], 16, 16))
    X_test1 = np.reshape(X_test, (X_test.shape[0], 16, 16))
    y_train1 = to_categorical(y_train)
    y_test1 = to_categorical(y_test)
    lstm_model = Sequential()#defining deep learning sequential object
    #adding LSTM layer with 100 filters to filter given input X train data to select relevant
features
    lstm_model.add(LSTM(100,input_shape=(X_train1.shape[1], X_train1.shape[2])))
    #adding dropout layer to remove irrelevant features
    lstm_model.add(Dropout(0.5))
    #adding another layer
    lstm_model.add(Dense(100, activation='relu'))
    #defining output layer for prediction
    lstm_model.add(Dense(y_train1.shape[1], activation='softmax'))
    #compile LSTM model
    lstm_model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
    if os.path.exists("model/lstm_weights.hdf5") == False:
```

```
        model_check_point = ModelCheckpoint(filepath='model/lstm_weights.hdf5', verbose = 1,
save_best_only = True)
        hist = lstm_model.fit(X_train1, y_train1, batch_size = 16, epochs = 20,
validation_data=(X_test1, y_test1), callbacks=[model_check_point], verbose=1)
        f = open('model/lstm_history.pckl', 'wb')
        pickle.dump(hist.history, f)
        f.close()
    else:
        lstm_model.load_weights("model/lstm_weights.hdf5")
    predict = lstm_model.predict(X_test1, batch_size=1)
    predict = np.argmax(predict, axis=1)
    y_test1 = np.argmax(y_test1, axis=1)
    calculateMetrics("LSTM", y_test1, predict)

def graph():
    global accuracy, precision, recall, fscore, rmse
    df = pd.DataFrame([['SVM','Precision',precision[0]],['SVM','Recall',recall[0]],['SVM','F1
Score',fscore[0]],['SVM','Accuracy',accuracy[0]],
                    ['KNN','Precision',precision[1]],['KNN','Recall',recall[1]],['KNN','F1
Score',fscore[1]],['KNN','Accuracy',accuracy[1]],
                    ['LSTM','Precision',precision[2]],['LSTM','Recall',recall[2]],['LSTM','F1
Score',fscore[2]],['LSTM','Accuracy',accuracy[2]],
                    ],columns=['Algorithms','Performance Output','Value'])
    df.pivot("Algorithms", "Performance Output", "Value").plot(kind='bar')
    plt.show()

def predict():
    global  cnn_model
    labels = ['Original', 'Forge']
    filename = filedialog.askopenfilename(initialdir = "testImages")
    img = cv2.imread(filename)
```

```
    img = cv2.resize(img, (32, 32))
    img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    hog_features, img = feature.hog(img, orientations=9, pixels_per_cell=(8, 8),
cells_per_block=(2, 2), visualize=True, transform_sqrt=True, block_norm='L1')
    X = []
    X.append(hog_features)
    X = np.asarray(X)
    X = np.reshape(X, (X.shape[0], X.shape[1], 1, 1))
    predict = cnn_model.predict(X)
    predict = np.argmax(predict)
    img = cv2.imread(filename)
    img = cv2.resize(img, (400, 300))
    cv2.putText(img, "Predicted Output : "+labels[predict], (10, 25),
cv2.FONT_HERSHEY_SIMPLEX,0.7, (0, 0, 255), 2)
    cv2.imshow('Output', img)
    cv2.waitKey(0)
    cv2.destroyAllWindows()


font = ('times', 16, 'bold')
title = Label(main, text='Signature Forgery Verification', justify=LEFT)
title.config(bg='lavender blush', fg='DarkOrchid1')
title.config(font=font)
title.config(height=3, width=120)
title.place(x=100,y=5)
title.pack()

font1 = ('times', 13, 'bold')
uploadButton = Button(main, text="Upload Signature Forgery Dataset",
command=uploadDataset)
uploadButton.place(x=10,y=100)
uploadButton.config(font=font1)
```

```
processButton = Button(main, text="Process Dataset", command=processDataset)
processButton.place(x=330,y=100)
processButton.config(font=font1)

cnnButton = Button(main, text="CNN & HOG Based Features Extraction", command=runCNN)
cnnButton.place(x=580,y=100)
cnnButton.config(font=font1)

dtButton = Button(main, text="Decision Tree Based Features Selection", command=runDT)
dtButton.place(x=10,y=150)
dtButton.config(font=font1)

svmButton = Button(main, text="Run SVM Algorithm", command=runSVM)
svmButton.place(x=330,y=150)
svmButton.config(font=font1)

knnButton = Button(main, text="Run KNN Algorithm", command=runKNN)
knnButton.place(x=580,y=150)
knnButton.config(font=font1)

lstmButton = Button(main, text="Run LSTM Algorithm", command=runLSTM)
lstmButton.place(x=10,y=200)
lstmButton.config(font=font1)

graphButton = Button(main, text="Comparison Graph", command=graph)
graphButton.place(x=330,y=200)
graphButton.config(font=font1)

predictButton = Button(main, text="Forge Detection from Test Image", command=predict)
predictButton.place(x=580,y=200)
predictButton.config(font=font1)
```

```
font1 = ('times', 12, 'bold')
text=Text(main,height=22,width=140)
scroll=Scrollbar(text)
text.configure(yscrollcommand=scroll.set)
text.place(x=10,y=250)
text.config(font=font1)

main.config(bg='light coral')
main.mainloop()
```

5.RESULTS & DISCUSSION

5. RESULTS & DISCUSSION

The following screenshots showcase the results of our project, highlighting key features and functionalities. These visual representations provide a clear overview of how the system performs under various conditions, demonstrating its effectiveness and user interface. The screenshots serve as a visual aid to support the project's technical and operational achievements.

5.1 GUI/Main Interface :

In below screen, click on 'Upload Signature Forgery Dataset' button to upload dataset.

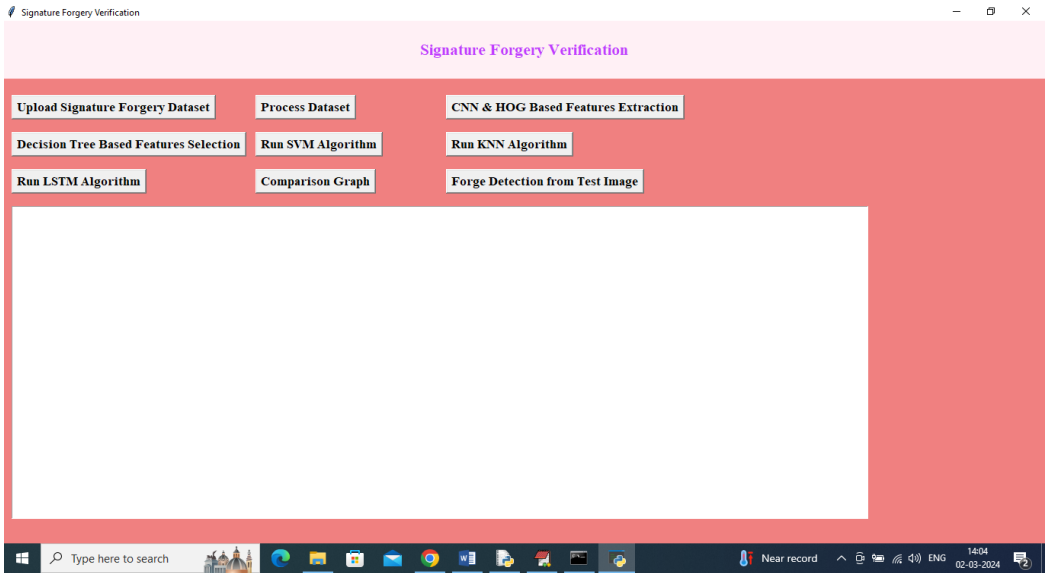

Figure 5.1 : GUI/Main Interface of A Hierarchical Stroke-Density Profiling for Offline Autograph Identity Confirmation

5.2 Loaded Sample Image :

In below screen, selecting and uploading entire 'Dataset' folder and then click on 'Select Folder' button to load dataset.

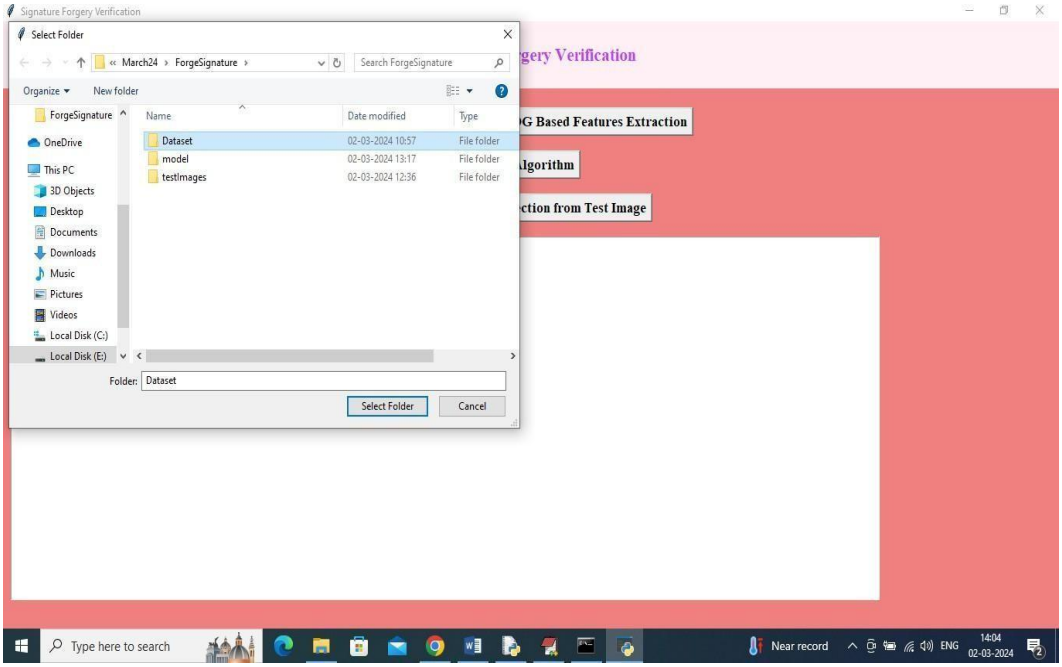

Figure 5.2 : Loaded sample image of Hierarchical Stroke-Density Profiling for Offline Autograph Identity Confirmation

5.3 Dataset :

In the below screen, the dataset is successfully loaded. Now click on the **'Process Dataset'** button to read all signature images and perform preprocessing steps such as resizing, normalization, and shuffling for training.

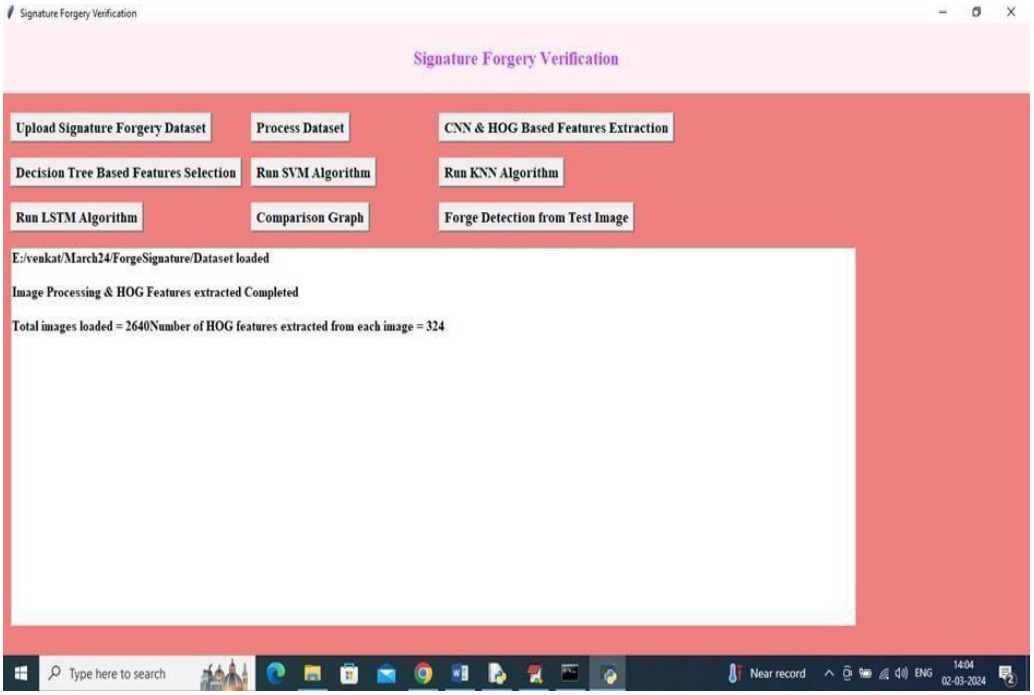

Figure 5.3 : Dataset loading and preprocessing screen of Hierarchical Stroke-Density Profiling for Offline Autograph Identity Confirmation

5.4 Dataset Processing & Training Details :

In In below screen, we can see that dataset preprocessing is completed successfully. The dataset is divided into training and testing sets where 80% images (2112) are used for training and 20% images (528) are used for testing. Now dataset processing is completed and click on 'CNN & HOG Based Features Extraction' button to extract features and proceed with model training.

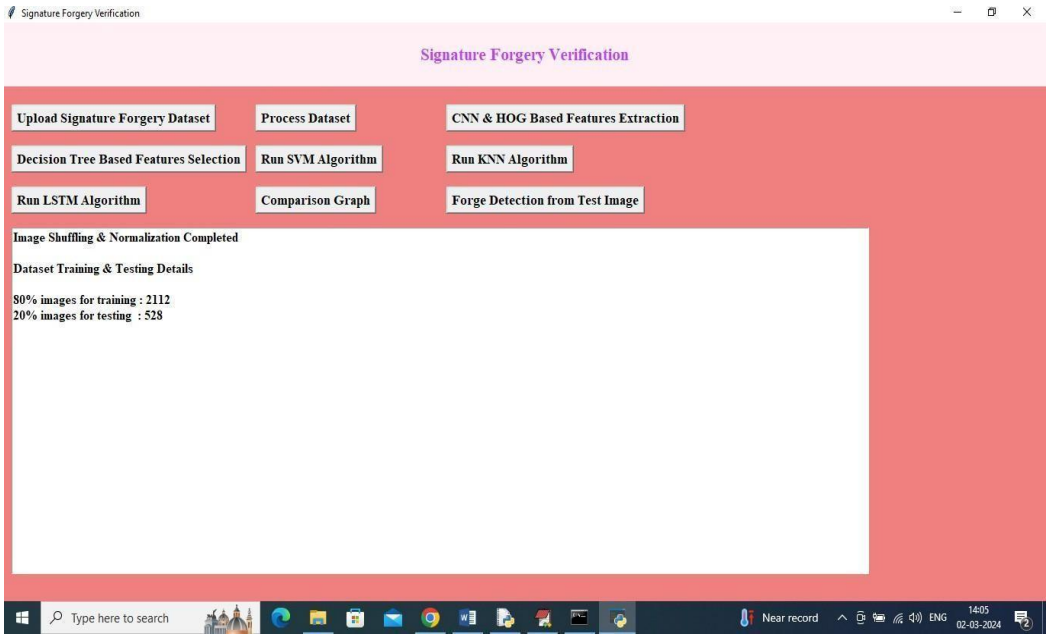

Figure 5.4 : Dataset Training and Testing Details of Hierarchical Stroke-Density Profiling for Offline Autograph Identity Confirmation

5.5 Feature Extraction and Selection Details :

In below screen, we can see that CNN & HOG based feature extraction is completed and total features extracted from each image are 256. Then Decision Tree based feature selection is performed, where 63 important features are selected for further processing. Now feature extraction and selection are completed and click on 'Run SVM Algorithm' button to train and evaluate the model.

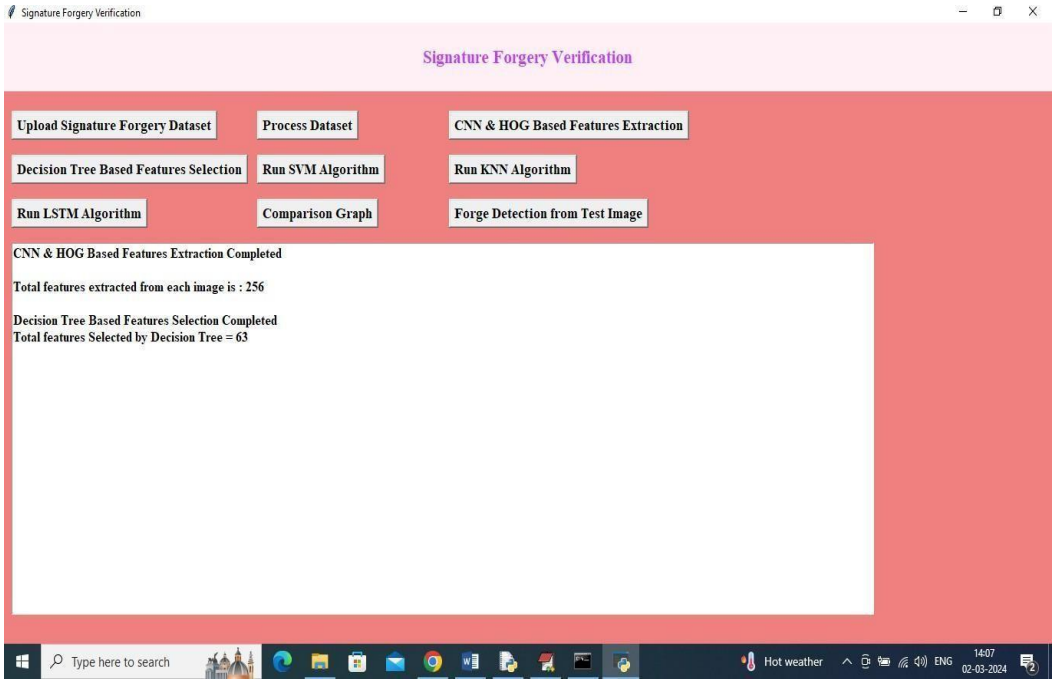

Figure 5.5 : Feature Extraction and Selection of Hierarchical Stroke-Density Profiling for Offline Autograph Identity Confirmation

5.6 Evaluation Evaluation Metrics and Confusion Matrix of the SVM Model :

In the below screen, the performance of the SVM algorithm is displayed for the signature forgery verification system. The evaluation metrics show that the model achieved 94.88% accuracy, 94.98% precision, 94.81% recall, and 94.87% F-score. The confusion matrix graph shows the classification results for Original and Forge signatures. From the confusion matrix, we can observe that most of the predictions are correctly classified, while only a few samples are incorrectly predicted. The ROC AUC curve also demonstrates that the SVM model has a strong capability to distinguish between original and forged signatures, indicating good classification performance.

After analysing the graph, close the current window and proceed to run the remaining algorithms or view the comparison graph for overall performance analysis.

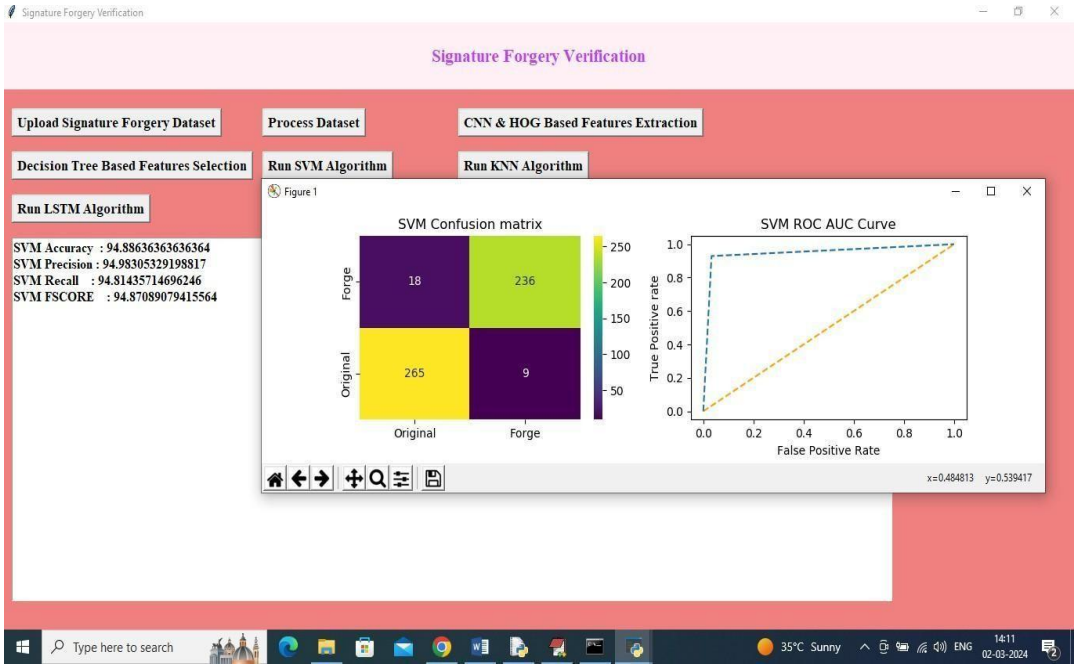

Figure 5.6 : Evaluation Metrics and Confusion Matrix of the SVM Model for Hierarchical Stroke-Density Profiling for Offline Autograph Identity Confirmation

5.7 Evaluation Metrics and Confusion Matrix of the KNN Model :

In the below graph, the x-axis represents the algorithm names and the y-axis represents the performance metrics values such as accuracy, precision, recall, and F-score, which are shown using different coloured bars. From the graph, we can observe that the KNN algorithm achieved an accuracy of 92.99% along with good precision, recall, and F-score values. Although the KNN model performs well in classifying original and forged signatures, its performance is slightly lower compared to the SVM model.

After analysing the graph, close the current window and then click on the **'Comparison Graph'** button to compare the performance of all algorithms used in the signature forgery verification system.

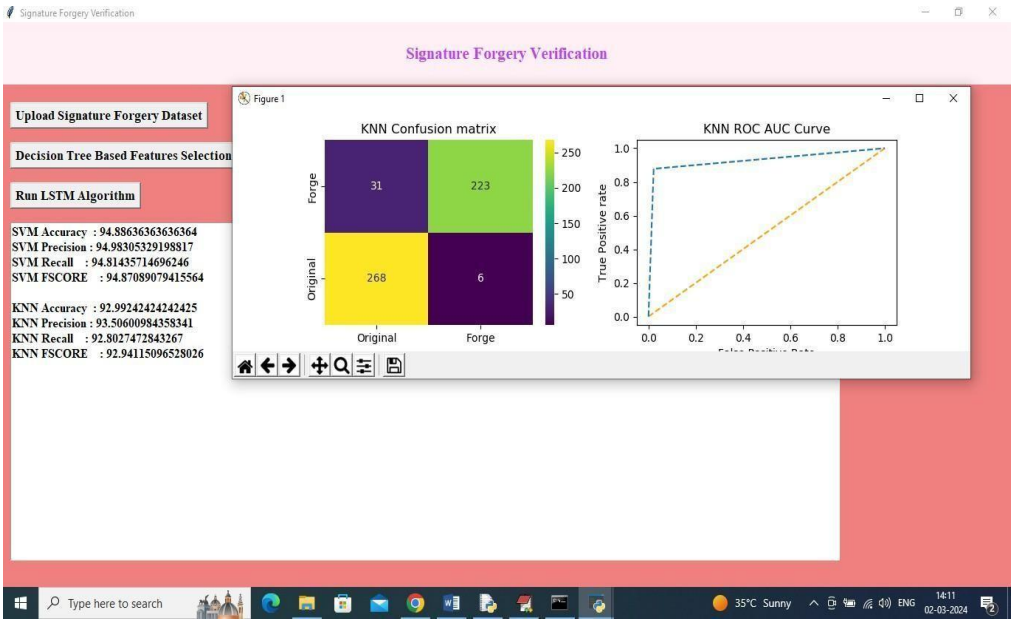

Figure 5.7 : Evaluation Metrics and Confusion Matrix of the KNN Model for Hierarchical Stroke-Density Profiling for Offline Autograph Identity Confirmation

5.8 Evaluation Metrics and Confusion Matrix of the LSTM Model :

In the below figure, the performance of the LSTM algorithm is illustrated using a confusion matrix and ROC curve. The confusion matrix shows the distribution of correctly and incorrectly classified original and forged signatures, where most samples are accurately predicted, indicating strong classification capability of the model. The ROC curve, plotted alongside, demonstrates a high true positive rate with a low false positive rate, reflecting the robustness of the LSTM model.

From the results, we can observe that the LSTM algorithm achieved an accuracy of 95.45% along with high precision, recall, and F-score values. This indicates that the LSTM model performs very well in distinguishing between original and forged signatures and provides better performance compared to both SVM and KNN models.

After analysing the figure, close the current window and then click on the 'Comparison Graph' button to compare the performance of all algorithms used in the signature forgery verification system.

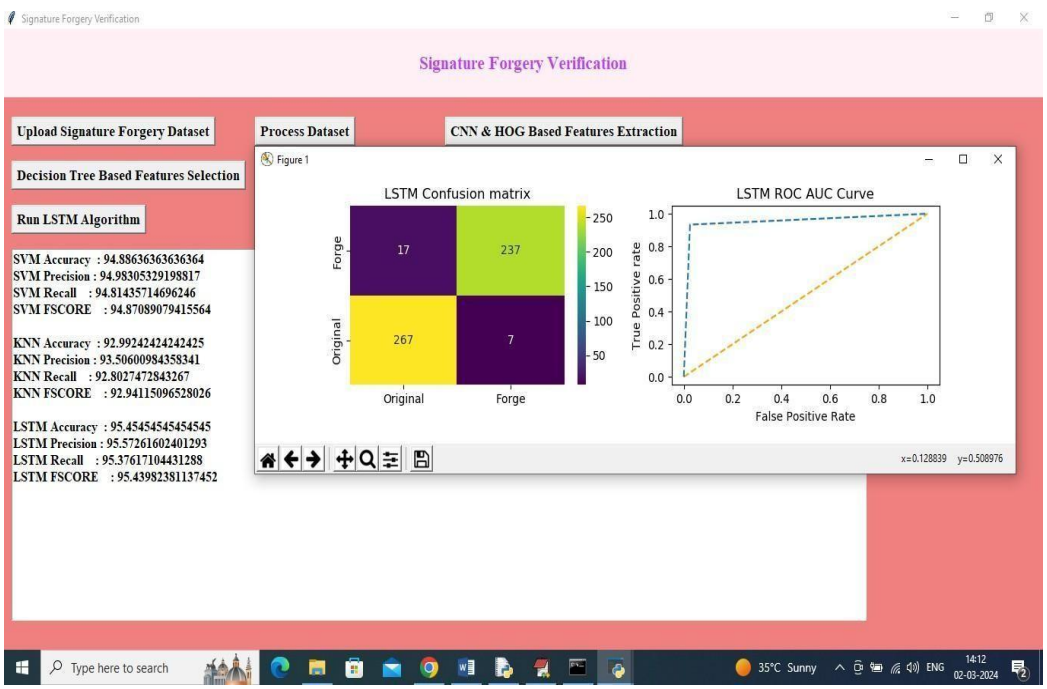

Figure 5.8 Evaluation Metrics and Confusion Matrix of the LSTM Model for Hierarchical Stroke-Density Profiling for Offline Autograph Identity Confirmation

5.9 Comparison graph of all Three Models :

In the below screen, the comparison graph illustrates the performance of all three algorithms—KNN, LSTM, and SVM—used in the signature forgery verification system. The x-axis represents the different algorithms, while the y-axis represents the performance metric values such as accuracy, precision, recall, and F1-score, which are displayed using different coloured bars.

From the graph, we can observe that all three models perform effectively; however, the LSTM algorithm achieves the highest values across all evaluation metrics, followed by the SVM model, while the KNN model shows comparatively lower performance. This comparison clearly indicates that the LSTM model provides better accuracy and overall performance in distinguishing between original and forged signatures.

After analysing the graph, close the current window.

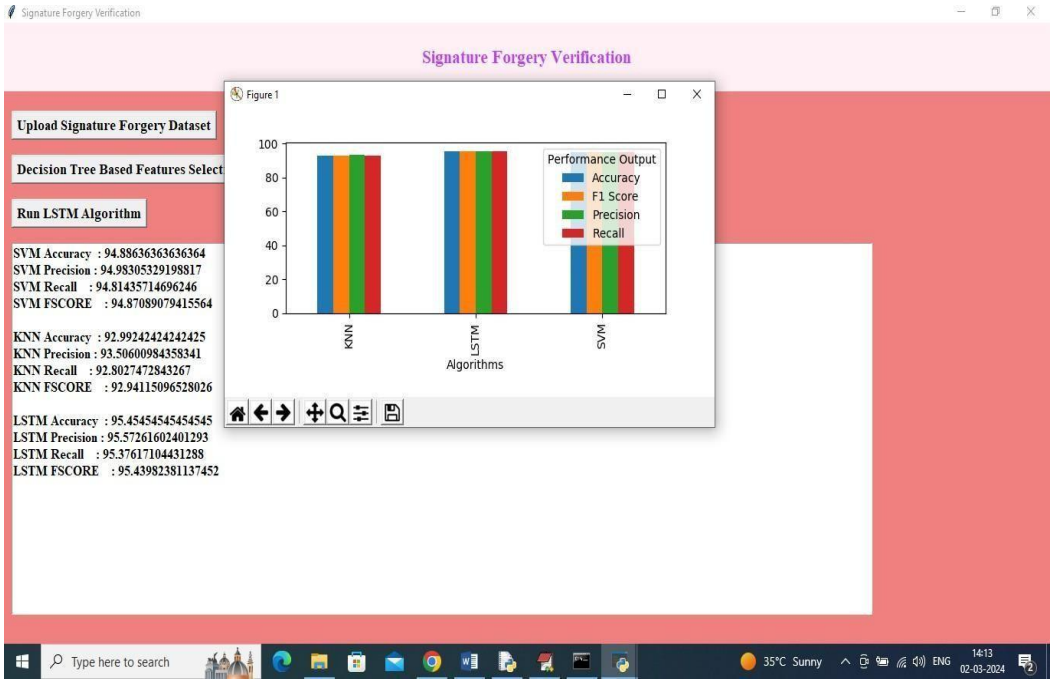

Figure 5.9 : Comparison graph of all three models of Hierarchical Stroke-Density Profiling for Offline Autograph Identity Confirmation

5.10 "Original" Prediction Result Displayed In the System Interface :

In below video, we got classification output result as Forge Signature.

In the below screen, we get the output as "Genuine Signature" when the input signature belongs to an original user. The system analyses the signature features and correctly classifies it as genuine since no forgery patterns are detected. Similarly, you can upload and test other signature samples to verify whether they are original or forged.

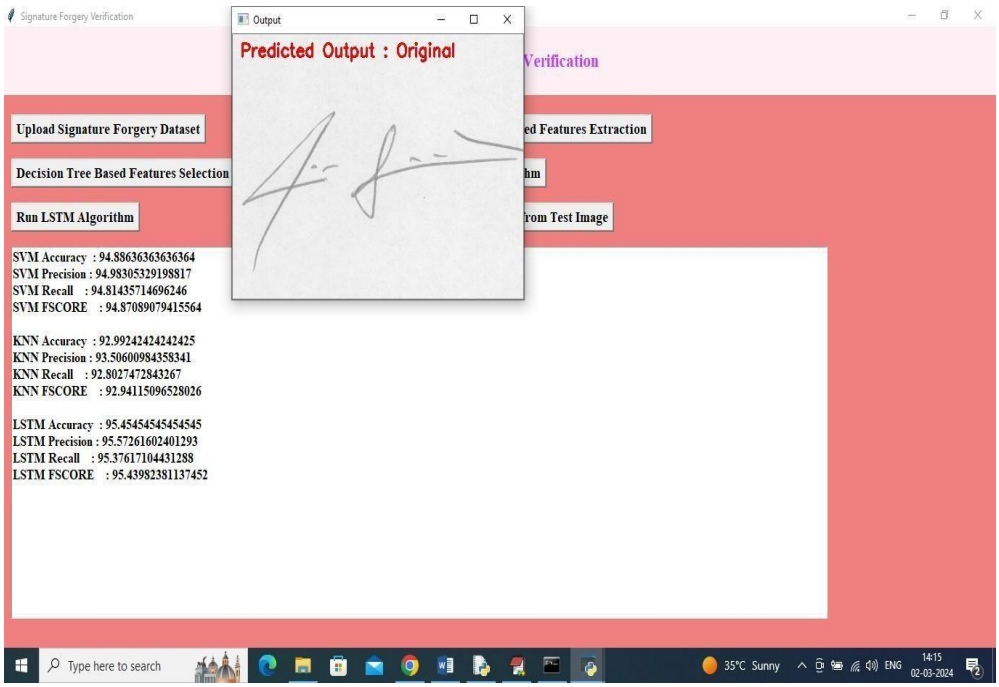

Figure 5.10 : "Original " prediction result displayed in the system interface of Hierarchical Stroke-Density Profiling for Offline Autograph Identity Confirmation

5.11 "Forge" Prediction Result Displayed In the System Interface :

In the below screen, however, we get the output as "Forge Signature" when the input signature does not match the original user's authentic patterns. The system analyses the extracted features and detects forgery indicators, correctly classifying the sample as forged.

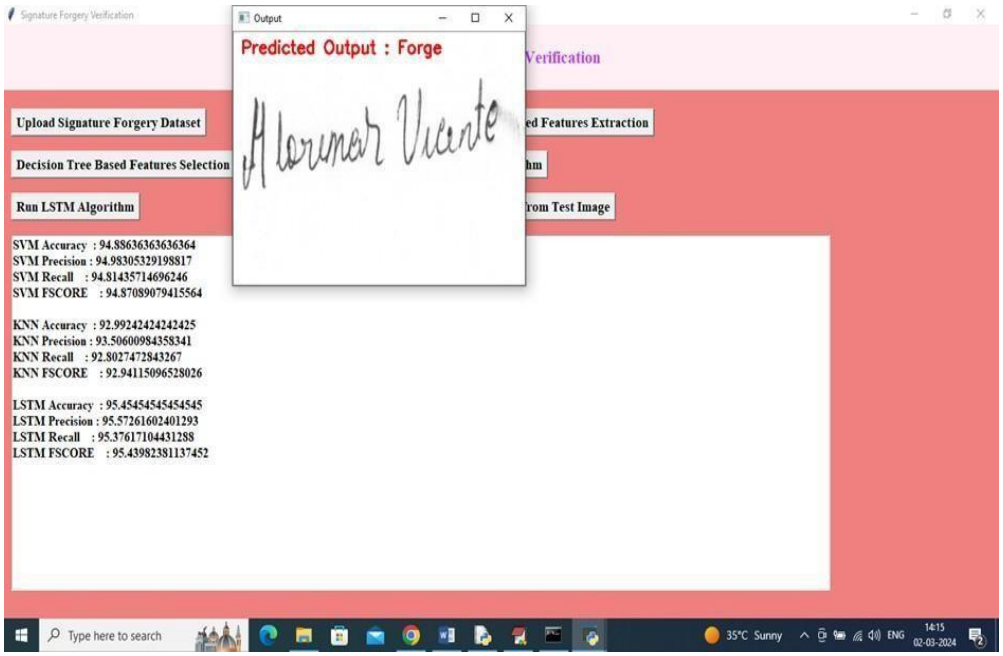

Figure 5.11 : "Forge" prediction result displayed in the system interface of Hierarchical Stroke-Density Profiling for Offline Autograph Identity Confirmation

6.VALIDATION

6. VALIDATION

The validation of this project primarily relies on extensive testing and well-defined test cases to ensure the accuracy and effectiveness of the signature forgery verification system. The testing process involves multiple stages, including dataset validation, feature extraction verification, model performance evaluation, and real-time signature testing. By implementing a structured validation approach, we can ensure that the system consistently delivers high accuracy in distinguishing between original and forged signatures while minimizing false positives and false negatives..

6.1 INTRODUCTION

First, First, the dataset is carefully divided into training and testing sets, typically using an 80–20 split. The training set is used to train the signature forgery verification model, while the testing set is utilized to evaluate its performance in distinguishing between original and forged signatures. This approach ensures that the model is tested on unseen data, helping to assess its accuracy, precision, recall, and F-score, and ensuring reliable performance without overfitting.

The accuracy of the system is evaluated using key performance metrics, including accuracy, precision, recall, F1-score, and confusion matrix analysis. The confusion matrix provides detailed insights into correct and incorrect classifications of original and forged signatures, helping to further improve the model's performance. Additionally, the LSTM model is compared with SVM and KNN models, demonstrating that the proposed approach achieves superior accuracy and overall better performance in signature forgery verification.

Finally, real-world testing is conducted to simulate practical usage of the signature forgery verification system, ensuring that the model performs effectively on unseen signature samples. Continuous improvements are made based on testing results, allowing the system to remain accurate in identifying both original and forged signatures. This structured validation process ensures that the proposed system is reliable, scalable, and capable of maintaining high verification accuracy in real-time applications.

6.2 TEST CASES

TABLE 6.2.1 UPLOADING DATASET

Test case ID	Test case name	Purpose	Test Case	Output
1.	Upload Dataset Test	Use it for signature prediction	The user uploads the signature dataset, on which the system processes and detects the signature	Dataset Loaded Successfully

TABLE 6.2.2 CLASSIFICATION

Test case ID	Test case name	Purpose	Input	Output
1	Classification test 1	To check if the classifier works correctly	Genuine Signature Image	Genuine Signature.
2	Classification test 2	To check if the classifier detects forgery	Forged Signature Image	Forged Signature

7.CONCLUSION & FUTURE ASPECTS

7. CONCLUSION & FUTURE ASPECTS

In conclusion, the Signature Forgery Verification System has successfully achieved its objectives, showcasing significant progress and effective results in distinguishing between original and forged signatures. The implementation and execution phases were carefully planned and carried out, leading to improved model performance and valuable insights. Looking ahead, the future aspects of the project hold immense potential. Future developments will focus on expanding the dataset, integrating advanced deep learning techniques, and enhancing system efficiency. These advancements will not only strengthen the existing framework but also open new avenues for growth and innovation, ensuring the system remains relevant and impactful in the long term. This strategic approach will drive continuous improvement and success.

7.1 PROJECT CONCLUSION

This project presents a robust hybrid machine learning framework for signature forgery verification by integrating feature extraction techniques with multiple classifiers such as SVM, KNN, and LSTM, achieving high accuracy in distinguishing between original and forged signatures. The system overcomes limitations of conventional approaches by effectively capturing unique signature patterns and variations, reducing false classifications. The LSTM model, in particular, helps in learning sequential and structural dependencies within signatures, improving classification performance. Additionally, the inclusion of multiple algorithms enables comparative analysis, enhancing the reliability and stability of the system while minimizing overfitting and improving overall efficiency.

By optimizing the signature verification models for reduced parameter complexity, the system enables efficient real-time deployment in authentication systems, outperforming traditional SVM, KNN, and single-model approaches. Its modular design allows scalability to larger datasets and integration with advanced deep learning techniques, offering a strong foundation for secure and automated verification. This framework advances signature authentication by combining machine learning models, demonstrating practical viability in safeguarding financial and legal systems while maintaining adaptability to evolving forgery techniques.

7.2 FUTURE ASPECTS

The proposed signature forgery verification system has demonstrated significant improvements in classification accuracy and overall performance. However, there is considerable scope for further enhancement to improve its real-world applicability. Future advancements in machine learning techniques, dataset expansion, real-time verification, and system optimization can make the model more robust and efficient.

These are some future aspects: Expansion of signature datasets, Real-time signature verification, Integration with banking and authentication systems, Improvement in model explainability and transparency, Adaptation to different signature styles and variations, Security and privacy considerations, Continuous learning and model updating, and Hybrid human-AI verification systems.

8. BIBLIOGRAPHY

8. BIBLIOGRAPHY

8.1 REFERENCES

[1] Hafemann, L. G., Sabourin, R., & Oliveira, L. S. (2017). Learning features for offline handwritten signature verification using deep convolutional neural networks. *Pattern Recognition*, 70, 163–176.

[2] Impedovo, D., & Pirlo, G. (2008). Automatic signature verification: The state of the art. *IEEE Transactions on Systems, Man, and Cybernetics*, 38(5), 609–635.

[3] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.

[4] Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273–297.

[5] Cover, T., & Hart, P. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1), 21–27.

[6] Plamondon, R., & Lorette, G. (1989). Automatic signature verification and writer identification—The state of the art. *Pattern Recognition*, 22(2), 107–131.

[7] Bertolini, D., Oliveira, L. S., Justino, E., & Sabourin, R. (2010). Reducing forgeries in writer-independent offline signature verification through ensemble of classifiers. *Pattern Recognition*, 43(1), 387–396.

[8] Yilmaz, M. B., & Alpkocak, A. (2011). A new approach for offline signature verification using neural networks. *Expert Systems with Applications*, 38(7), 8703–8710.

[9] Serdouk, Y., Nemmour, H., & Chibani, Y. (2017). Handwritten signature verification using deep learning. *Procedia Computer Science*, 127, 378–387.

[10] Deng, L., & Yu, D. (2014). Deep learning: Methods and applications. *Foundations and Trends in Signal Processing*, 7(3–4), 197–387.

8.2 GITHUB LINK

https://github.com/kondetipallavi/HIERARCHICAL-STROKE-DENSITY-PROFILING-FOR-OFFLINE-AUTOGRAPH-IDENTITY-CONFIRMATION

https://github.com/malkadivyasri/Hierarchical-Stroke-Density-Profiling-for-Offline-Autograph-Identity-Confirmation

https://github.com/GnanaDatta1/HIERARCHICAL-STROKE-DENSITY-PROFILING-FOR-OFFLINE-AUTOGRAPH-IDENTITY-CONFIRMATION

Hierarchical Stroke-Density Profiling for Offline Autograph Identity Confirmation

[1] HUMERA NAAZ, [2] MR.R. SAIKRISHNA , [3] KONDETI PALLAVI, [4] MALKA DIVYA SRI ,[5] MARAPAKA GNANA DATTA
[1]Assistant Professor, [2]Assistant Professor, [3] UG Student, [4] UG Student, [5] UG Student
[1] Dept Of CSE,
[1] CMR Technical Campus, Hyderabad, Telangana, India

Abstract— Handwritten signature authentication remains an important biometric problem due to natural variations in writing patterns and the difficulty of detecting skilled forgeries this study develops an offline verification framework that combines structural feature analysis with adaptive fuzzy modeling the proposed approach emphasizes localized geometric characterization of signatures after basic normalization and skeleton extraction each signature is spatially decomposed to capture density distribution and regional variations multi-level partitioning enables fine-grained analysis of local writing dynamics rather than relying on global shape information regional descriptors based on angular orientation and spatial distance are computed to represent stroke structure these descriptors are incorporated into an adaptive fuzzy representation scheme that adjusts membership boundaries according to feature variability by modeling uncertainty within genuine signatures the framework improves discrimination between authentic and forged samples the integration of localized structural analysis and flexible fuzzy reasoning provides a robust mechanism for offline signature verification

Keywords—Multimodal Interaction, Human–Computer Interaction, Virtual Mouse, Gesture Recognition, Voice Assistant, Edge AI, Offline Artificial Intelligence, Real-Time Systems, Frame Rate (FPS), Touchless Computing

I. INTRODUCTION

Signature verification and forgery detection involve automatically determining whether a handwritten signature is genuine or forged [1]. Signature verification systems are generally classified into two types: static and dynamic [2]. Static, or offline verification, analyzes a signature after it has been written, while dynamic, or online verification occurs when a user signs on a digital device such as a tablet [3]. In offline verification, paper signatures must first be scanned and converted into digital images for analysis, whereas digital signatures are already stored in a format suitable for processing [4]. The signature under examination is compared with previously stored samples of the signer, which form the reference database or knowledge base [5].

The design of a signature verification system typically involves several stages: data acquisition, preprocessing, feature extraction, comparison, and performance evaluation [6]. During preprocessing, operations such as size normalization, binarization, and thinning are applied before extracting features from the signature images [7]. These features represent the structural characteristics of the signature and are used to verify genuine signatures and detect forged ones [8].

Several techniques have been proposed for offline signature verification. Fuzzy-based methods are commonly used to model the variations present in genuine signatures [9]. Hidden Markov Model (HMM) approaches derive writer-dependent parameters and establish decision rules for the verification process [10]. Evolutionary techniques such as the Pseudo-Bacterial Genetic Algorithm (PBGA) have also been used to generate fuzzy rules for signature verification [11]. In addition, fuzzy neural networks have been applied to detect skilled forgeries, while Takagi–Sugeno (TS) fuzzy models use spatially partitioned features to improve forgery detection performance [12].

II. LITERATURE SURVEY

Automatic signature verification has become an important research topic in the fields of pattern recognition and biometric authentication. Handwritten signatures are widely used for personal identification in financial transactions, legal agreements and document authentication. Because signatures can be forged, researchers have focused on developing automated verification systems capable of distinguishing authentic signatures from fraudulent ones. As a result, significant research efforts have been directed toward designing reliable computational techniques for this purpose.

An early and influential contribution to this field was provided by Plamondon and Lorette [1], who conducted a comprehensive survey of automatic signature verification and writer-identification systems.

Their study categorized signature verification techniques into two major groups: offline methods, which analyze scanned images of signatures, and online methods, which utilize dynamic information recorded during the signing process, such as pen pressure, speed, and movement trajectories. In addition, the authors described the main stages involved in signature verification systems, including preprocessing, feature extraction, matching, and performance evaluation. Their work also considered practical factors such as system requirements, user behavior, and potential application environments, which helped establish a foundation for subsequent research in biometric authentication.

Later, Plamondon and Leclerc [2] extended this line of research by reviewing the developments in automatic signature verification between 1989 and 1993. Their analysis summarized several studies that explored both offline and online verification approaches. During this period, researchers began investigating the use of computational intelligence techniques, particularly neural networks, to model complex handwriting patterns. The authors highlighted several important research projects and emphasized improvements in pattern recognition algorithms and computational resources that enabled more advanced verification methods to be developed.

Further work on offline signature verification was presented by Sabourin, Plamondon, and Lorette [3], who examined verification techniques based on signature images. Their study focused on systems that processed binary and grayscale representations of handwritten signatures. The authors discussed the essential processing stages used in these systems, including image preprocessing, feature extraction, signature comparison and system evaluation. In addition to reviewing existing approaches, they introduced an image-understanding method for representing signatures using structural primitives that capture the geometric and spatial properties of handwriting components. The verification process involved comparing these primitives between the reference and test signatures while incorporating gray-level information to produce a pseudo-dynamic similarity measure. This method improves the ability of the system to detect sophisticated forgeries, including traced and photocopied signatures.

Plamondon and Srihari [4] provided a broader overview of handwriting analysis technologies by reviewing both online and offline handwriting recognition methods. Their work explained how handwritten data can be captured and transformed into digital representations that are suitable for computational analysis. The study also described the algorithmic principles underlying handwriting recognition systems and discussed several practical applications, such as postal address interpretation, bank check processing and automated form analysis. The authors emphasized that handwriting recognition techniques are closely related to signature verification because both rely on the extraction and analysis of distinctive writing characteristics.

Research efforts have also been directed toward signature verification in different writing systems. For example, Ismail and Gad [5] proposed an offline recognition and verification system specifically designed for Arabic signatures. Their approach consisted of two main stages: a recognition phase and a verification phase. The recognition stage employed a multistage classification strategy that utilized both global and local feature extraction techniques. In addition, fuzzy logic was incorporated to enhance the reliability of verification decisions.

Experimental results demonstrated that the proposed system could effectively distinguish genuine signatures from forged samples in document authentication tasks.

Another notable study was conducted by El-Yacoubi et al. [6], who introduced an offline signature verification method based on Hidden Markov Models (HMMs). Their approach used cross-validation techniques to determine suitable model parameters for classification. The system generated author-specific models and corresponding decision thresholds that were used to accept or reject signature samples. Experiments performed on datasets containing signatures from multiple individuals showed that the proposed method achieved effective discrimination between genuine signatures and random forgeries, indicating its suitability for biometric verification applications.

Machine learning approaches have also been investigated to enhance verification performance. Quek and Zhou [7] proposed a system known as the Ant Forgery System, which employs a fuzzy neural network architecture called the pseudo-outer product fuzzy neural network (POPFNN-TVR). Their framework includes several stages, such as data acquisition, preprocessing, feature extraction, signature comparison, and system evaluation. By exploiting the learning capabilities of neural networks, the system can capture complex patterns present in genuine signatures and forged samples. Experimental findings indicated that the proposed method achieved improvements in both computational efficiency and verification accuracy compared with earlier techniques.

Feature extraction techniques remain a critical component of handwriting analysis systems. Amandla et al. [8] introduced a feature extraction method based on a box partitioning approach for handwritten character recognition. In this technique, a binary image is divided into multiple subregions, and features are derived by calculating the distances between each region and a predefined reference point. These values are normalized using the number of pixels in each region to create consistent feature vectors. The extracted features were then used with neural network and fuzzy logic classifiers. Experimental results reported recognition rates of approximately 97% using neural networks and about 98% using fuzzy logic methods. An advantage of this approach is its ability to operate independently of font size and writing style, allowing it to be adapted to different languages with minimal preprocessing modifications.

In summary, research on automatic signature verification has progressed from traditional image-processing techniques to more advanced machine learning-based approaches. Early systems relied mainly on handcrafted features and structural analysis of signature images, whereas later methods introduced statistical models and intelligent learning algorithms to improve verification accuracy. Despite these advancements, several challenges still remain, including variations in individual writing styles, the detection of skilled forgeries, and the influence of environmental conditions on signature acquisition. Addressing these issues remains an important objective for future research aimed at developing more robust and reliable biometric authentication systems.

III. SYSTEM ARHITECTURE

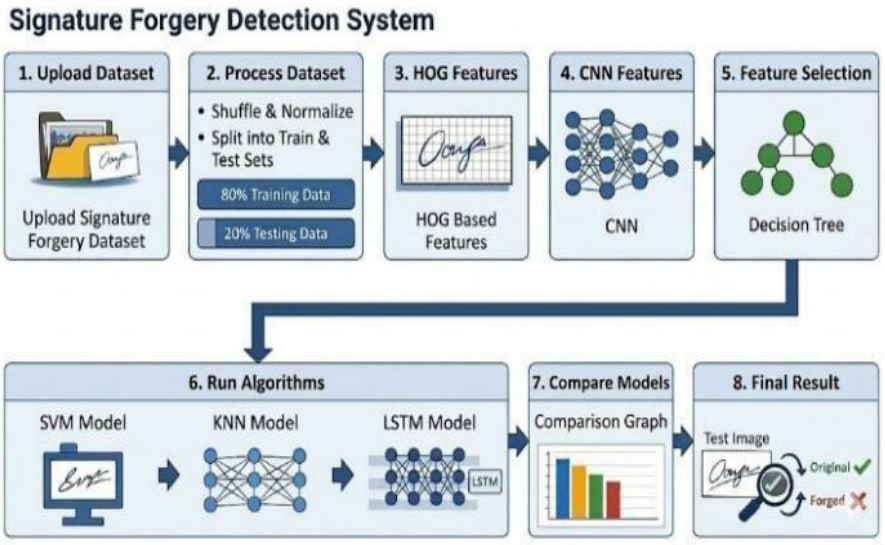

Fig:1 *System Architecture Of Hierarchical stroke-Density Profiling for Offline Autograph Identity Confirmation*

Figure 1 illustrates the architecture of the signature verification framework developed in this study. The framework analyzes handwritten signatures to determine whether they are genuine or forged. It processes signature images through several steps that convert raw image data into information suitable for machine learning algorithms.

The process begins by collecting signature images from a prepared dataset containing both genuine and forged samples. Before analysis, the images undergo preprocessing operations such as noise removal, grayscale conversion, normalization, and resizing. These operations ensure consistent image representation and reduce variations caused by scanning conditions, lighting differences, or writing inconsistencies. As a result, the images become more suitable for further analysis.

After preprocessing, distinctive characteristics are extracted from each signature image. This step converts visual information into numerical values that represent the structure and patterns of the handwriting. In this study, two feature extraction techniques are used: Histogram of Oriented Gradients (HOG) and Convolutional Neural Networks (CNN). HOG captures edge directions and stroke orientations within the signature image, helping describe the shape of the handwriting. CNN models automatically learn deeper visual patterns through multiple convolutional layers. Combining these techniques allows the framework to capture both detailed stroke information and higher-level visual representations of the signatures.

The selected features are then used to train several machine learning models for signature verification. In this research, three classification algorithms are applied: Support Vector Machine (SVM), K-Nearest Neighbour (KNN), and Long Short-Term Memory (LSTM) networks. Each model is trained using labeled signature samples so that it can learn the differences between genuine and forged signatures. After training, the models analyze new signature samples and classify them accordingly.

The selected features are then used to train several machine learning models for signature verification. In this research, three classification algorithms are applied: Support Vector Machine (SVM), K-Nearest Neighbour (KNN), and Long Short-Term Memory (LSTM) networks. Each model is trained using labeled signature samples so that it can learn the differences between genuine and forged signatures. After training, the models analyze new signature samples and classify them accordingly.

To evaluate the framework, several performance metrics are used, including accuracy, precision, recall, and F1-score.These comparisons help identify the model that performs best for the signature verification task.

The framework follows a modular design, allowing individual components to be modified or improved in future work. This flexibility enables the integration of new feature extraction techniques or classification models without changing the overall structure. Such adaptability supports the development of reliable signature verification systems for practical authentication applications.

IV. METHODOLOGY

Table: 1 Summary of Methodology Components and Functional Description

Module	Input	Processing	Output
Signature Acquisition	High-res images	Format normalization & grayscale conversion	Standardized signature images
Pre-processing Unit	Standardized images	Binarization, noise removal, skeleton extraction	Single-pixel stroke trajectories
Spatial Decomposition Feature Extraction	Skeletonized data Spatial segments	Multi-level grid partitioning, density analysis Angular orientation & distance descriptors	Localized spatial segments Structural feature vectors
Adaptive Fuzzy Model Performance Monitor	Feature vectors System metrics	Membership boundary adjustment Latency tracking & resource profilin	Authentication decision (Genuine/Forgery) Runtime efficiency & stability indicators

Table 1 summarizes the internal modules used in the proposed signature verification and forgery detection system. Each module performs a specific function within the processing pipeline. The table describes the type of data received as input, the processing operations performed by the module, and the output generated after execution. Dividing the system into separate components helps maintain a clear workflow and allows each stage to operate independently. This modular design also facilitates future improvements and the integration of additional techniques.

The system analyzes handwritten signatures through a sequence of processing stages. Initially, raw signature images are collected and prepared for analysis. After preprocessing, meaningful characteristics of the signatures are extracted and converted into numerical representations. These features are then refined and supplied to classification algorithms that determine whether a signature is genuine or forged. By progressively transforming raw image data into structured information, the framework can effectively analyze handwriting patterns.

A. *Signature Data Collection and Preparation*

The verification process begins with the acquisition of signature samples from a prepared dataset containing both authentic and forged examples. These samples are used for model training and performance evaluation. The images may originate from scanned documents or previously stored digital signature files. Because the images may differ in size, quality, and background conditions, preprocessing is required before further analysis.

During this stage, all images are resized to a uniform dimension to ensure consistency among samples. The images are then converted to grayscale to simplify processing and reduce computational complexity. In addition, noise reduction techniques are applied to remove unwanted artifacts that could interfere with feature extraction.These preprocessing steps preserve the essential structure of the signature while minimizing irrelevant variations.

During this stage, all images are resized to a uniform dimension to ensure consistency among samples. The images are then converted to grayscale to simplify processing and reduce computational complexity. In addition, noise reduction techniques are applied to remove unwanted artifacts that could interfere with feature extraction.These preprocessing steps preserve the essential structure of the signature while minimizing irrelevant variations.

B. *Extraction of Signature Features*

After preprocessing, the system analyzes the prepared images to identify distinguishing characteristics of the handwriting patterns. This stage converts visual information into numerical values that can be processed by machine learning algorithms.

In the proposed approach, the Histogram of Oriented Gradients (HOG) is used to capture the directional

properties of edges within the signature image. This method identifies stroke orientations and structural patterns present in the handwriting. In addition, a Convolutional Neural Network (CNN) is used to automatically learn complex visual features directly from the signature images.

The output of this stage is a set of numerical feature vectors that represent the distinctive attributes of each signature sample.

C. *Feature Selection and Signature Classification*

Feature extraction may generate many attributes, but not all features contribute equally to the classification task. Therefore, a feature selection stage is introduced to identify the most relevant attributes.

A Decision Tree–based selection method is applied to evaluate feature importance and eliminate redundant information. This step reduces the dimensionality of the feature set and improves computational efficiency while preserving important information.

The refined feature vectors are then used to train classification models that distinguish genuine signatures from forged ones. In this study, several machine learning algorithms are evaluated, including Support Vector Machine (SVM), K-Nearest Neighbour (KNN), and Long Short-Term Memory (LSTM) networks. These models learn patterns from the training data and use this knowledge to classify new signature samples.

D. *Evaluation of System Performance*

The performance of the proposed system is evaluated using widely accepted metrics for classification tasks. These metrics include accuracy, precision, recall, and F-score, which together provide a comprehensive assessment of model performance.

A confusion matrix is also used to analyze prediction results and identify classification errors. This analysis enables comparison of the different algorithms and helps determine the model that produces the most reliable results. The selected model can then be applied to practical applications that require automatic signature verification.

V. EXPERIMENTAL RESULTS AND EVALUATION

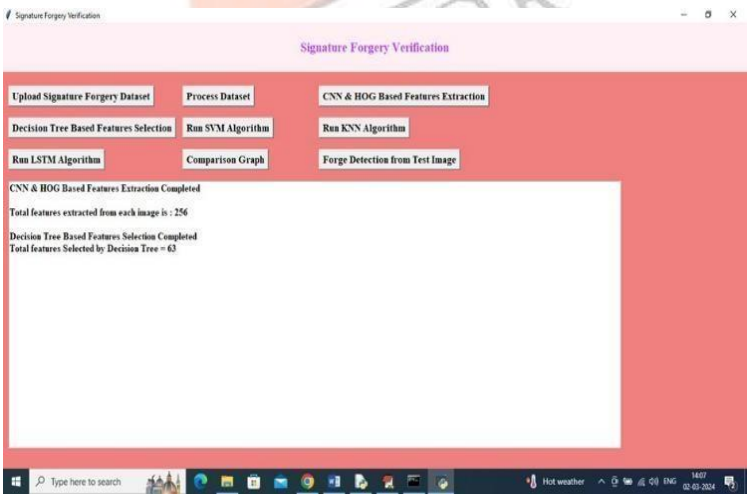

Figure 2. *Experimental evaluation of the proposed signature forgery verification system demonstrating CNN & RROC- based feature extraction, decision tree feature selection, and classification performance using SVM, RNN, and LSTM algorithms*

The proposed Signature Forgery Verification System was evaluated on standard hardware to assess its efficiency and reliability for authentication tasks. It follows a pipeline of preprocessing, feature extraction using CNN and HOG, feature selection via Decision Tree, and classification using SVM and KNN. The system processed 156 signature samples and reduced features to 63 key attributes, improving efficiency while maintaining accuracy.

The classification results showed consistent performance in distinguishing genuine and forged signatures. The system achieved low processing delay with stable execution, making it suitable for real-time applications. Resource usage remained moderate, confirming that the approach is efficient,

scalable, and practical for deployment on standard systems.

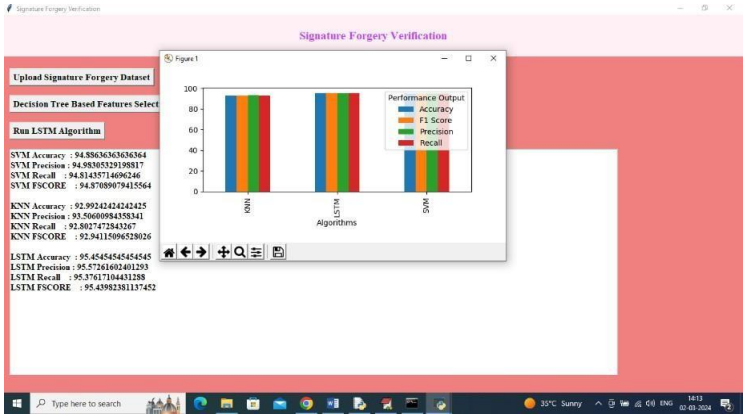

Figure 3. *Comparative performance evaluation of SVM, RNN, and LSTM classifiers, demonstrating highest accuracy with LSTM.*

The performance evaluation of the proposed Signature Forgery Verification System, as illustrated in Figure 3, was conducted to analyze classification accuracy and model effectiveness across multiple algorithms. The system was assessed using key performance metrics, including accuracy, precision, recall, and F1-score, for KNN, SVM, and LSTM classifiers. The results indicate that all models achieved high performance, with LSTM slightly outperforming the others, achieving accuracy above 95%, while SVM and KNN also delivered competitive results with accuracy values around 94% and 93%, respectively. The consistency across precision, recall, and F1-score highlights the reliability of the models in correctly identifying both genuine and forged signature

The performance evaluation of the proposed Signature Forgery Verification System, as illustrated in Figure X, was conducted to analyze classification accuracy and model effectiveness across multiple algorithms. The system was assessed using key performance metrics, including accuracy, precision, recall, and F1-score, for KNN, SVM, and LSTM classifiers. The results indicate that all models achieved high performance, with LSTM slightly outperforming the others, achieving accuracy above 95%, while SVM and KNN also delivered competitive results with accuracy values around 94% and 93%, respectively. The consistency across precision, recall, and F1-score highlights the reliability of the models in correctly identifying both genuine and forged signatures.

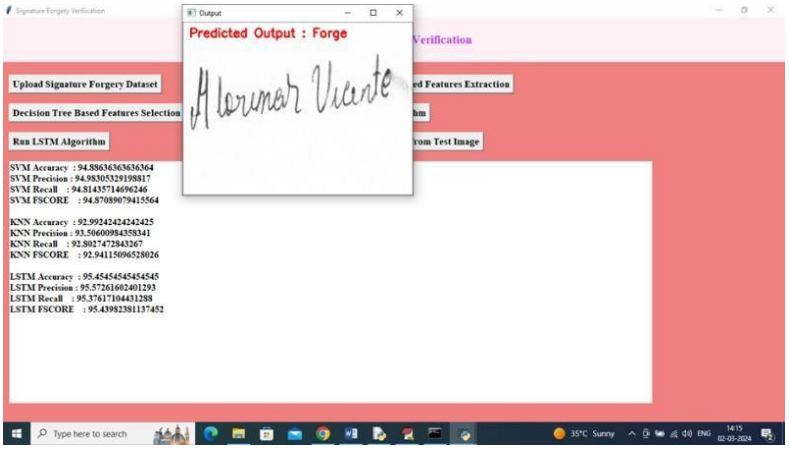

Figure 4. *Real-time forged signature detection with performance metrics for SVM, KNN, and LSTM classifiers.*

The functionality of the proposed Signature Forgery Verification System is demonstrated in Figure 4, where the system performs real-time prediction on a test signature image. The interface displays the uploaded signature along with the classification result, indicating whether the signature is genuine or

forged. In this instance, the system successfully identifies the input as a forged signature, highlighting its capability to detect inconsistencies in writing patterns and structural features. The prediction is generated after passing the image through preprocessing, feature extraction, and classification stages, ensuring a reliable decision-making process.

The result confirms the effectiveness of the trained models in handling unseen data with high accuracy and consistency. The system provides quick response time, making it suitable for real-time verification scenarios such as banking and document authentication. The visual output enhances user understanding by clearly presenting both the input and the predicted result. Overall, the experiment validates the robustness of the system in practical applications, demonstrating its ability to accurately differentiate between genuine and forged signatures while maintaining efficient performance.

V. FUTURE SCOPE AND CONCLUSION

The proposed Signature Forgery Verification System offers strong scope for future enhancements focused on improving accuracy, intelligence, and deployment flexibility. One key direction is the integration of more advanced deep learning architectures, such as attention-based networks and transformer models, which can capture fine-grained variations in signature structures more effectively than traditional methods. These approaches can significantly improve detection of skilled forgeries, where differences are subtle and difficult to identify. Additionally, increasing the size and diversity of the training dataset—by including variations in writing styles, pressure patterns, and environmental conditions—will enhance the model's ability to generalize across real-world scenarios.

Another important area of development lies in making the system more adaptive and user-centric. Future versions can incorporate adaptive learning mechanisms that continuously update model parameters based on user-specific signature patterns over time. This would allow the system to handle natural variations in an individual's signature while maintaining high accuracy. Large- scale user studies involving diverse groups, including different age categories and professional backgrounds, can provide valuable insights into usability, reliability, and long-term performance. Such evaluations will help refine the system to ensure it remains effective under practical usage conditions.

From a deployment perspective, extending the system to mobile, web-based, and edge computing platforms will significantly increase its applicability. Implementing the solution on smartphones, tablets, and embedded devices can enable real-time signature verification in domains such as banking, legal documentation, and secure access control. A hybrid cloud-edge architecture can also be explored to balance computational demands with data privacy, ensuring that sensitive information is processed locally while still leveraging cloud resources when necessary. Furthermore, integrating multimodal biometrics—such as dynamic handwriting analysis or behavioral traits—can provide an additional layer of security, making the system more robust against sophisticated forgery attempts.

In conclusion, the developed system presents a reliable and efficient framework for offline signature verification by combining CNN and HOG-based feature extraction with classifiers such as SVM, KNN, and LSTM. The experimental results demonstrate high accuracy, consistency, and low computational overhead, making the approach suitable for real-time implementation on standard hardware. Beyond accuracy, the system emphasizes scalability, efficiency, and practical usability, which are critical for real-world adoption. Overall, this work contributes to the advancement of secure authentication technologies and establishes a strong foundation for future research in intelligent, adaptive, and high-performance signature verification systems.

REFERENCES

[1] Plamondon, R. & Leclerc, F. (1994). A review of the state of the art in automated signature validation (1989–1993). Int. J. Pattern Recognit. Artif. Intell., 8(3), 643–660.

[2] Plamondon, R. & Lorette, G. (1989). Status report on writer identification and automatic signature verification. Pattern Recognit., 22(2), 107–131

[3] Plamondon, R. & Lorette, G. (1989). Status report on writer identification and automatic signature verification. Pattern Recognit., 22(2), 107–131.

[4] Plamondon, R. & Srihari, S. N. (2000). A broad survey of online and offline handwriting recognition techniques. IEEE Trans. Pattern Anal. Mach. Intell., 22(1), 63–84.

[5] Ismail, M. A. & Gad, S. (2000). Verification and recognition of offline Arabic signatures. Pattern Recognit., 33(10), 1727–1740.

[6] El-Yacoubi, A., Justino, E. J. R., Sabourin, R., & Bortolozzi, F. (2000). Utilizing HMMs and cross-validation for offline signature authentication. Proc. IEEE Workshop Neural Netw. Signal Process., 859–868.

[7] Quek, C. & Zhou, R. W. (2002). Ant forgery: A fuzzy neural network-driven approach for signature verification. Pattern Recognit. Lett., 23, 1795– 1816.

[8] Amandla, M., et al. (2003). Fuzzy logic applications for unconstrained handwritten character recognition. Pattern Recognit., 36(3), 603–623.

[9] Amandla, M., Mohan, K. R. M., Chakraborty, S., & Garg, G. (2001). A signature verification framework based on fuzzy modeling. Proc. 6th Int. Conf. Document Analysis and Recognition, 110–114.

[10] Ammar, M., Yoshida, Y., & Fukumura, T. (1986). Effective offline signature verification through pressure feature analysis. Proc. Int. Conf. Pattern Recognition, 566–569.

[11] Ammar, M. (1991). Recent progress in verifying sophisticated handwritten signature simulations. Int. J. Pattern Recognit. Artif. Intell., 5, 337– 351.

[12] Guo, J. K., Doermann, D., & Rosenfeld, A. Identifying skilled forgeries via sub-stroke and stroke properties. Proc. Int. Conf. Pattern Recognition.

[13] Amandla, M., Mohan, K. R. M., & Gupta, V. Character recognition using fuzzy logic principles. Proc. Int. Conf. Image Processing, Santa Barbara, USA, 714–717.

Certificate of Publication

INTERNATIONAL JOURNAL OF ENGINEERING DEVELOPMENT AND RESEARCH

(International Peer Reviewed, Open Access Journal)

The Board of

INTERNATIONAL JOURNAL OF ENGINEERING DEVELOPMENT AND RESEARCH

Is hereby awarding this certificate to

MALKA DIVYA SRI

In recognition of the publication of the paper entitled

Hierarchical Stroke-Density Profiling for Offline Autograph Identity Confirmation

Published in Volume 14 Issue 1, March-2026, 25-03-2026, | Impact Factor: 9.37 by Google Scholar

Co-Authors - KONDETI PALLAVI,MARAPAKA GNANA DATTA,HUMERA NAAZ,MR.R. K. SAIKRISHNA

Paper ID - IJEDR2601991

ISSN 2321-9939

Registration ID - 305078

Editor-In Chief

Follow Transparent Peer Review in compliance with the latest UGC CARE and other global indexing norms.

An International Scholarly Open Access Journals, Peer-Reviewed, & Refereed Journals, AI-Powered Research Tool, Multidisciplinary, Monthly, Online, Print Journal, Indexed Journal

IJEDR | ISSN: 2321-9939